Redes Neurais e Deep Learning





Prof. Rodrigo Caropreso
Jun/2019
rcaropreso at gmail

Esta Foto de Autor Desconhecido está licenciado em CC BY-SA

Conteúdo Programático – Aulas 1 e 2

Introdução e Redes Multi-Camadas

- Conceitos Gerais em Aprendizado Profundo
- Perceptron Multi-Camadas;
- Backpropagation e Validação Cruzada.
- Redes Neurais com Retardos Temporais (TDNN);

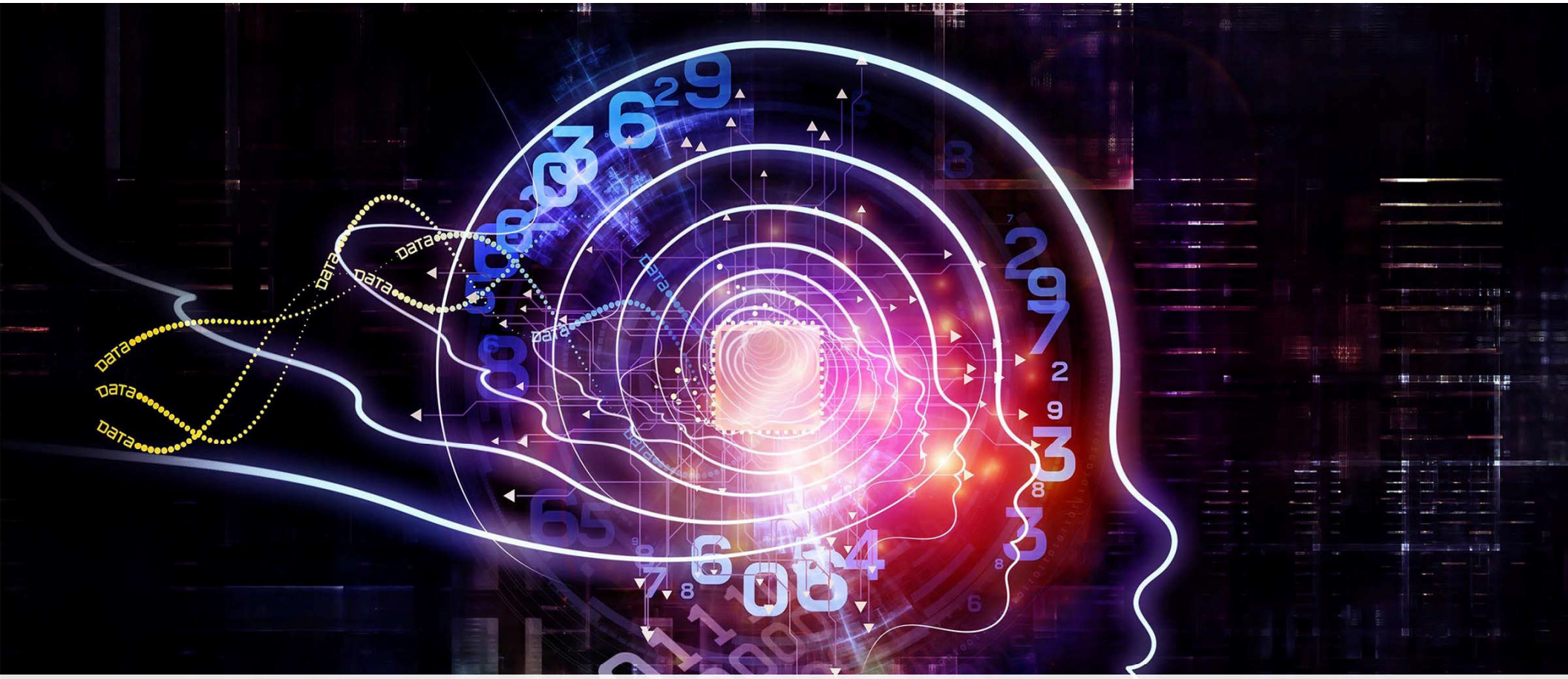
Redes Neurais Recorrentes

- Redes de Jordan
- Redes de Elman
- Pytorch – Conceitos e Exercícios
- Redes Convolucionais: Conceitos e Exercícios

Conteúdo Programático – Aula 3

Redes Neurais Convolucionais

- TensorFlow: Conceitos e Exercícios
- Visão Computacional: Visão Geral
- Redes AlexNet: Conceitos e Exercícios
- Redes LSTM: Conceitos e Exercícios



Perceptron Multi-Camadas (PMC)

Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Perceptron Multi-Camadas: Conceitos



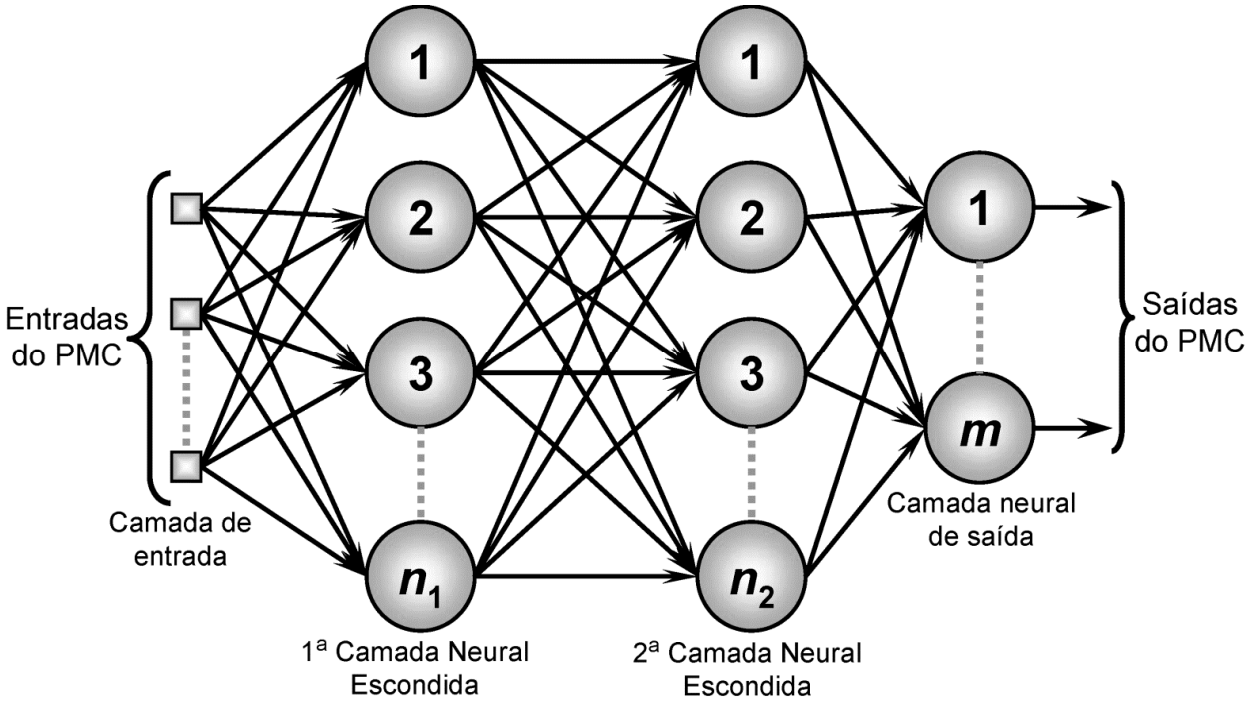
- Redes Perceptron de Múltiplas Camadas (PMC), também conhecidas como redes MLP (*Multi Layer Perceptron*), são caracterizadas pela presença de pelo menos uma camada intermediária (escondida) de neurônios.
- O PMC pertence à arquitetura feedforward de camadas múltiplas.
- O treinamento do PMC é executado de forma SUPERVISIONADA.

Perceptron Multi-Camadas: Conceitos



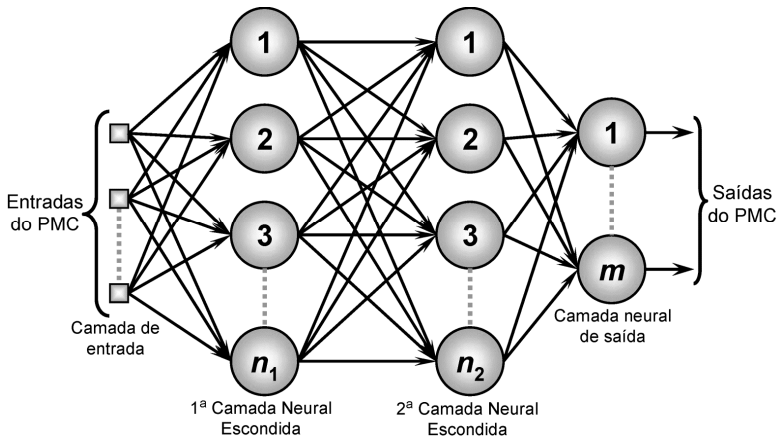
- É uma das mais versáteis quanto às suas aplicações, podendo ser utilizadas nos seguintes tipos de problemas:
  - Aproximação universal de funções.
  - Classificação de padrões.
  - Identificação e controle de processos.
  - Previsão de séries temporais.
  - Otimização de sistemas.

Perceptron Multi-Camadas: Arquitetura



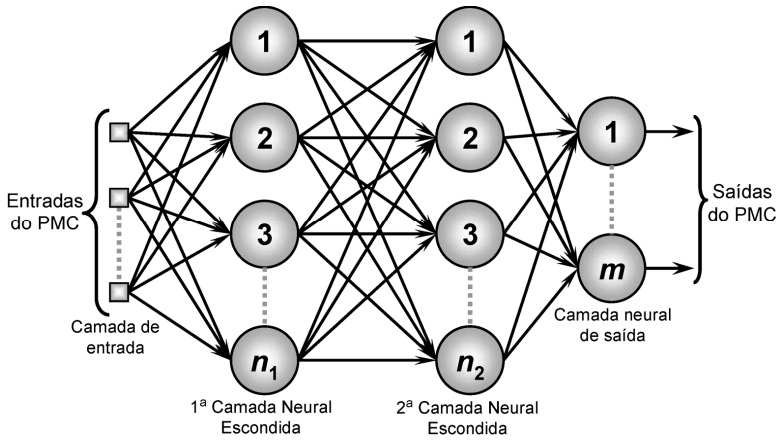
- É uma das mais versáteis quanto às suas aplicações, podendo ser utilizadas nos seguintes tipos de problemas:
- No PMC convencional **inexiste** qualquer tipo de realimentação de valores produzidos pela camada neural de saída ou pelas próprias camadas neurais intermediárias.

Perceptron Multi-Camadas: Funcionamento



- As entradas do PMC, representando os sinais advindos de determinada aplicação, será propagada camada-a-camada em direção à sua camada neural de saída.
- As saídas dos neurônios da primeira camada neural de saída serão as próprias entradas daqueles neurônios pertencentes à segunda camada neural escondida.
- As saídas dos neurônios da segunda camada neural escondida serão as respectivas entradas dos neurônios pertencentes à sua camada neural de saída.

Perceptron Multi-Camadas: Funcionamento



- Ao contrário do Perceptron e ADALINE, a camada neural de saída do PMC pode ser composta por diversos neurônios:
- Cada um destes neurônios de saída representaria uma das saídas do processo a ser mapeado.
- As camadas intermediárias, por sua vez, extraem a maioria das informações referentes ao seu comportamento e as codificam por meio dos pesos sinápticos e limiares de seus neurônios.

Perceptron Multi-Camadas: Treinamento



1. **Primeira Fase (Forward):**
   - Os sinais $\{x_1, x_2, \dots, x_n\}$ de uma amostra de treinamento são inseridos nas entradas da rede.
   - Estes são propagados camada-a-camada até a produção das respectivas saídas.
   - Leva-se em consideração apenas valores atuais de pesos sinápticos e limiares de seus neurônios, os quais permanecerão inalterados durante cada execução desta fase.
   - A aplicação desta fase visa tão somente obter as respostas da rede.

Perceptron Multi-Camadas: Treinamento



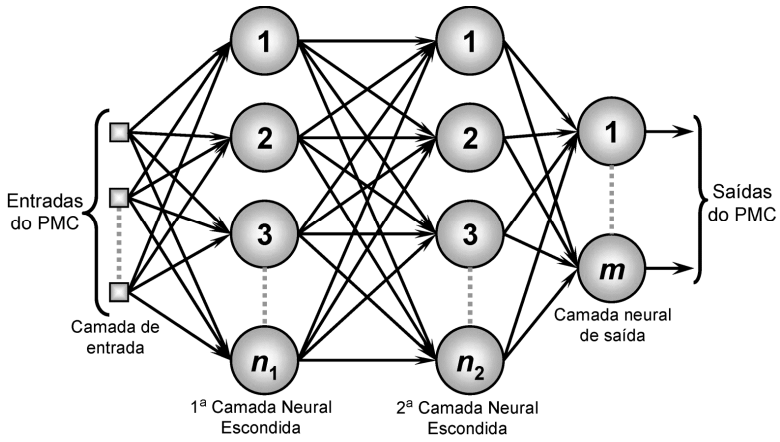
2. **Análise de Erro (Função custo):**

   - As respostas produzidas pelas saídas do PMC são comparadas com as respectivas respostas desejadas (aprendizado supervisionado).

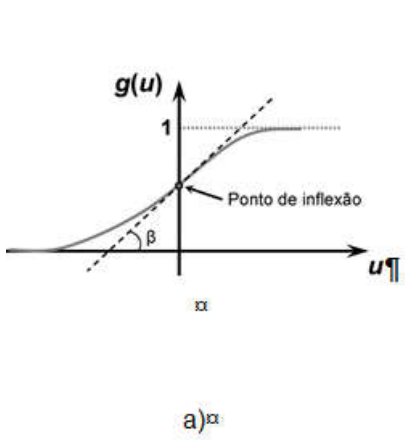
3. **Segunda Fase (Backpropagation) :**

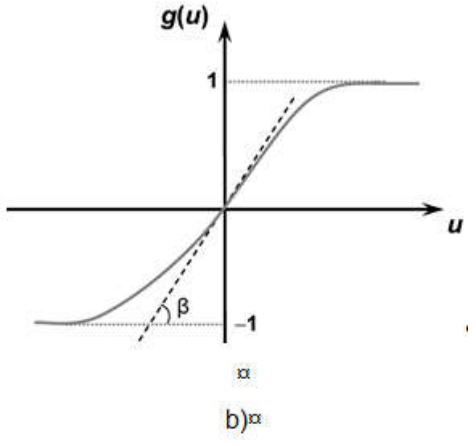
   - Baseados nos desvios (erros) entre às respostas desejadas e àquelas produzidas pelos neurônios de saída, ajustam-se os pesos e limiares dos neurônio do PMC.
   - A aplicação desta fase visa então ajustar pesos e limiares de todos os neurônios.

Perceptron Multi-Camadas: Funções de Ativação



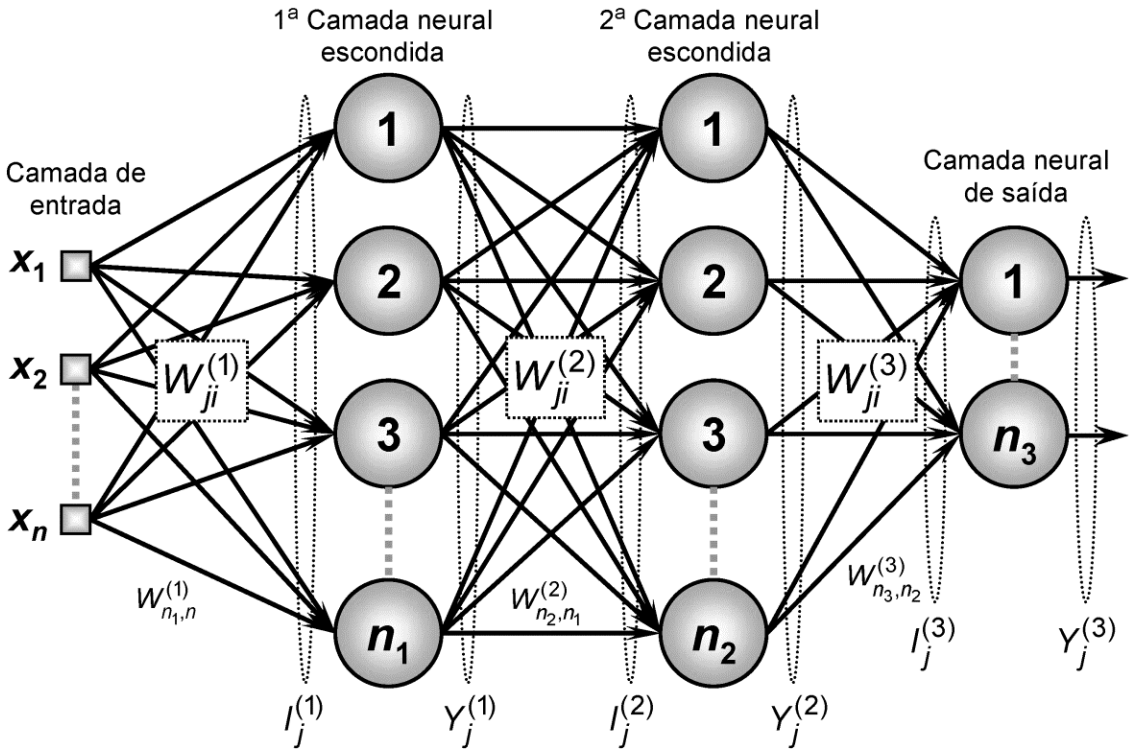
- Função Logística/Tangente Hiperbólica





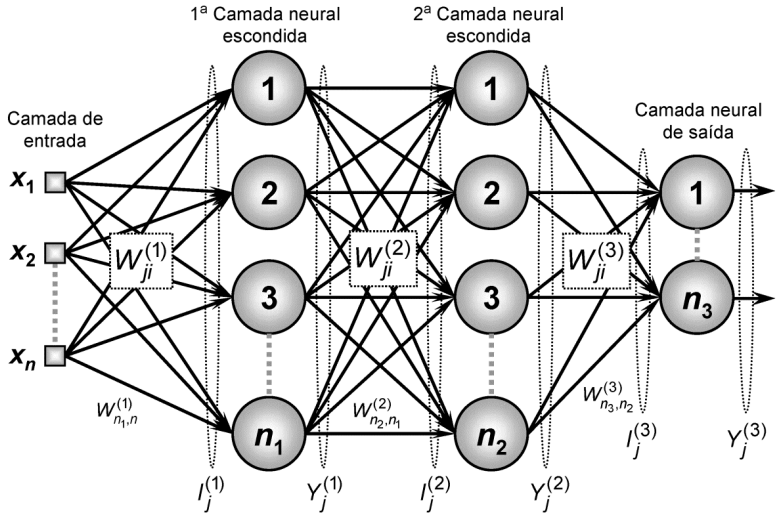
$$g(u) = \frac{1}{1 + e^{-\beta u}} \qquad g(u) = \frac{1 - e^{-\beta u}}{1 + e^{-\beta u}}$$

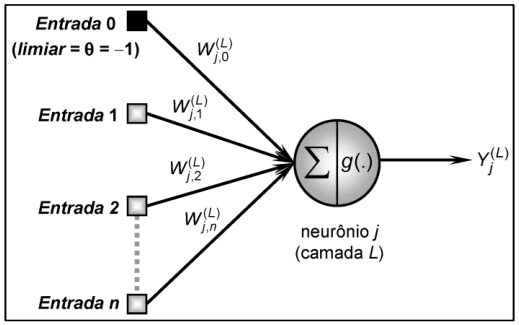
Perceptron Multi-Camadas: Notação



- $W_{ji}^{(L)}$ são matrizes de pesos cujos elementos denotam o valor do peso conectando o j-ésimo neurônio da camada (L) ao i-ésimo neurônio da camada (L-1). Para a topologia Ilustrada, tem-se:
  - $W_{ji}^{(3)}$ é o peso sináptico conectando o j-ésimo neurônio da camada de saída ao i-ésimo neurônio da camada 2.
  - $W_{ji}^{(2)}$ é o peso sináptico conectando o j-ésimo neurônio da camada escondida 2 ao i-ésimo neurônio da camada 1.
  - $W_{ji}^{(1)}$é o peso sináptico conectando o j-ésimo neurônio da camada 1 ao i-ésimo sinal da camada de entrada

Perceptron Multi-Camadas: Notação





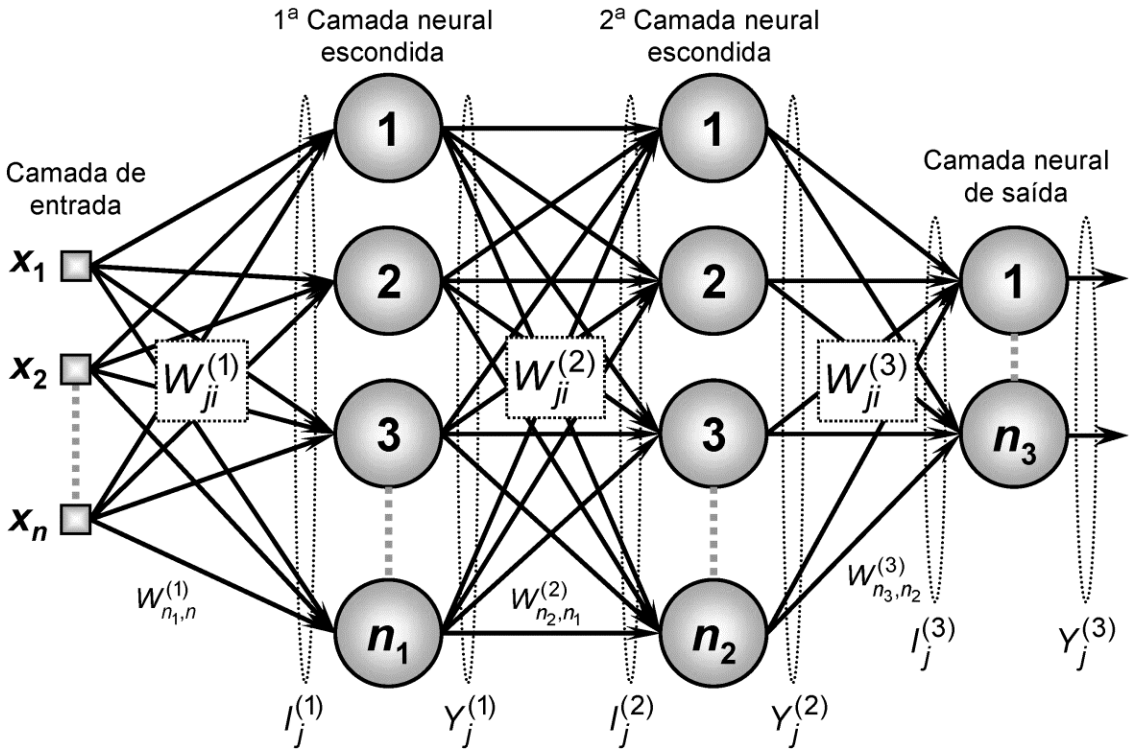
- $I_j^{(L)}$ são vetores cujos elementos denotam a entrada ponderada em relação ao j-ésimo neurônio da camada L, os quais são definidos por:

$$I_j^{(1)} = \sum_{i=0}^{n} W_{ij}.x_i \rightarrow I_j^{(1)} = W_{1,0}^{(1)}.x_0 + W_{1,1}^{(1)}.x_1 + \cdots + W_{1,n}^{(1)}.x_n$$

$$I_j^{(2)} = \sum_{i=0}^{n1} W_{ij}.x_i \rightarrow I_j^{(2)} = W_{1,0}^{(2)}.x_0 + W_{1,1}^{(2)}.x_1 + \cdots + W_{1,n}^{(2)}.x_{n1}$$

$$I_j^{(3)} = \sum_{i=0}^{n2} W_{ij}.x_i \rightarrow I_j^{(3)} = W_{1,0}^{(3)}.x_0 + W_{1,1}^{3}.x_1 + \cdots + W_{1,n}^{(3)}.x_{n2}$$

Perceptron Multi-Camadas: Notação



- $Y_i^{(L)}$ são vetores cujos elementos denotam a saída do j-ésimo neurônio em relação à camada L, os quais são definidos por:

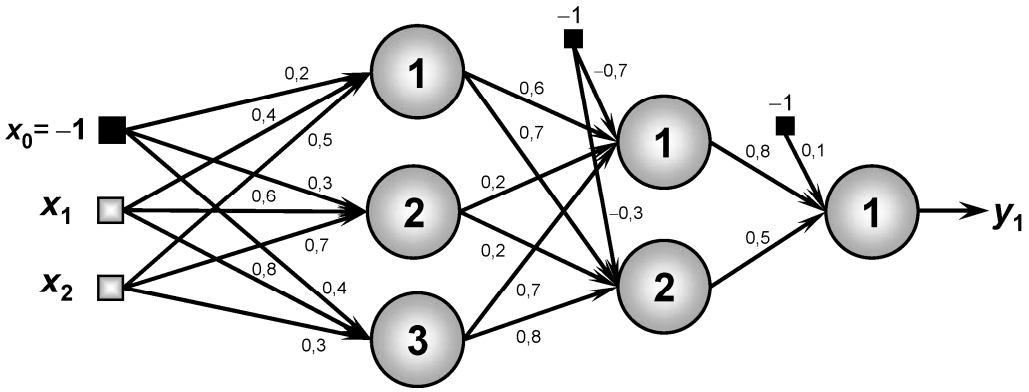
$$Y_j^{(1)} = g(I_j^{(1)})$$

$$Y_j^{(2)} = g(I_j^{(2)})$$

$$Y_j^{(3)} = g(I_j^{(3)})$$

- O funcional $g(.)$ representa uma função de ativação que deve ser contínua e diferenciável em todo o seu domínio, tais como a função de ativação logística ou tangente hiperbólica.

Perceptron Multi-Camadas: Funções de Ativação



$$x = \begin{bmatrix} -1 \\ 0.3 \\ 0.7 \end{bmatrix}$$

- Dados do PMC:

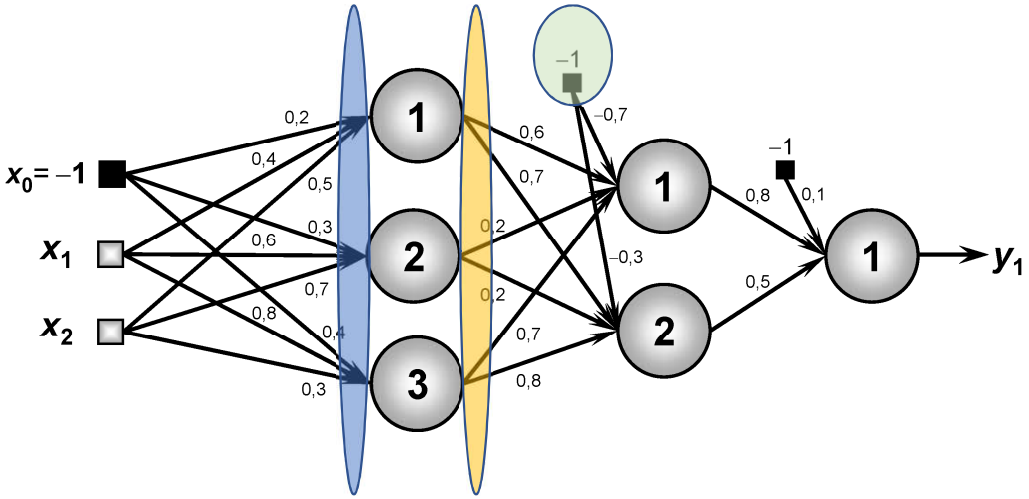
$$n_1 = 3 \quad n_2 = 2 \quad n_3 = 1$$

$$W_{ji}^{(1)} = \begin{bmatrix} 0.2 & 0.4 & 0.5 \\ 0.3 & 0.6 & 0.7 \\ 0.4 & 0.8 & 0.3 \end{bmatrix}$$

$$W_{ji}^{(2)} = \begin{bmatrix} -0.7 & 0.6 & 0.2 & 0.2 \\ -0.3 & 0.7 & 0.2 & 0.8 \end{bmatrix}$$

$$W_{ji}^{(3)} = [0.1 \quad 0.8 \quad 0.5]$$

Perceptron Multi-Camadas: Funções de Ativação



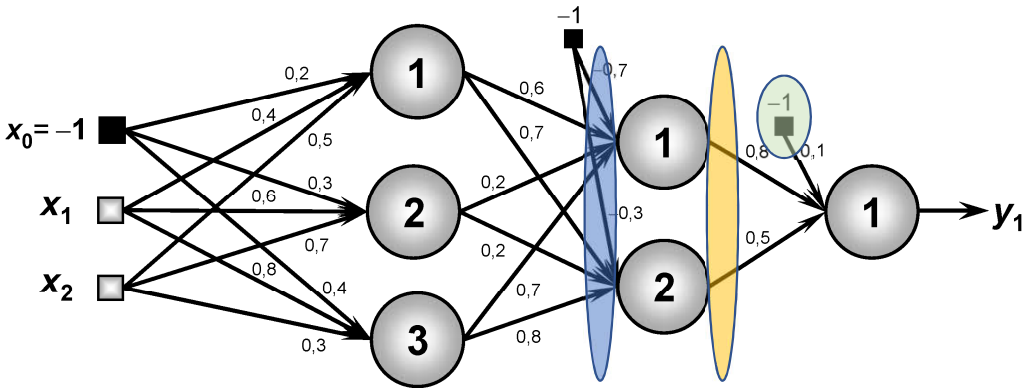
- Camada 1:

$$I_j^{(1)} = W_{ji}^{(1)} \cdot x = \begin{bmatrix} 0.2 & 0.4 & 0.5 \\ 0.3 & 0.6 & 0.7 \\ 0.4 & 0.8 & 0.3 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ 0.3 \\ 0.7 \end{bmatrix} = \begin{bmatrix} 0.27 \\ 0.37 \\ 0.05 \end{bmatrix}$$

$$Y_j^{(1)} = g_1\left(I_j^{(1)}\right) = \begin{bmatrix} g_1\left(I_1^{(1)}\right) \\ g_1\left(I_2^{(1)}\right) \\ g_1\left(I_3^{(1)}\right) \end{bmatrix} = \begin{bmatrix} \tanh(0.27) \\ \tanh(0.35) \\ \tanh(0.05) \end{bmatrix} = \begin{bmatrix} 0.26 \\ 0.35 \\ 0.05 \end{bmatrix}$$

$$Y_j^{(1)} = \begin{bmatrix} -1 \\ 0.26 \\ 0.35 \\ 0.05 \end{bmatrix} \leftarrow Y_0^{(1)} = -1$$

Perceptron Multi-Camadas: Funções de Ativação



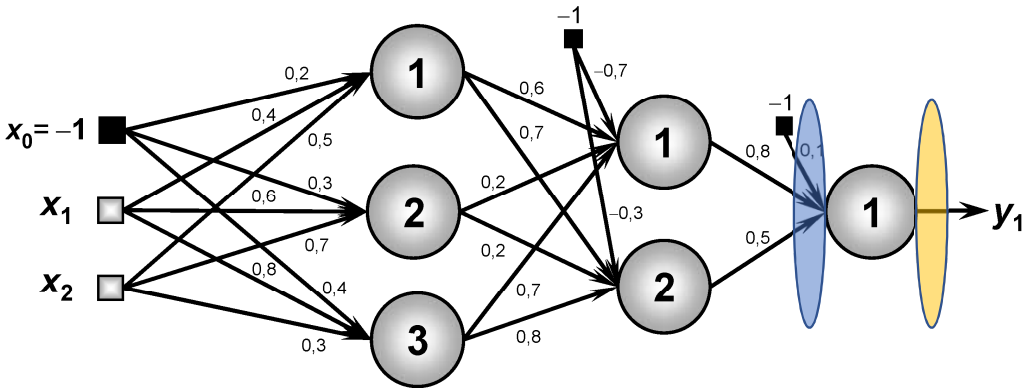
- Camada 2:

$$I_j^{(2)} = W_{ji}^{(2)} \cdot I_j^{(1)} = \begin{bmatrix} -0.7 & 0.6 & 0.2 & 0.2 \\ -0.3 & 0.7 & 0.2 & 0.8 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ 0.26 \\ 0.35 \\ 0.05 \end{bmatrix} = \begin{bmatrix} 0.96 \\ 0.59 \end{bmatrix}$$

$$Y_j^{(2)} = g_2\left(I_j^{(2)}\right) = \begin{bmatrix} g_2\left(I_1^{(2)}\right) \\ g_2\left(I_2^{(2)}\right) \end{bmatrix} = \begin{bmatrix} \tanh(0.96) \\ \tanh(0.59) \end{bmatrix} = \begin{bmatrix} 0.74 \\ 0.53 \end{bmatrix}$$

$$Y_j^{(2)} = \begin{bmatrix} -1 \\ 0.74 \\ 0.53 \end{bmatrix} \leftarrow Y_0^{(2)} = -1$$

Perceptron Multi-Camadas: Funções de Ativação



- Camada 3:

$$I_j^{(3)} = W_{ji}^{(3)} \cdot I_j^{(2)} = [0.1 \quad 0.8 \quad 0.5] \cdot \begin{bmatrix} -1 \\ 0.74 \\ 0.53 \end{bmatrix} = [0.76]$$

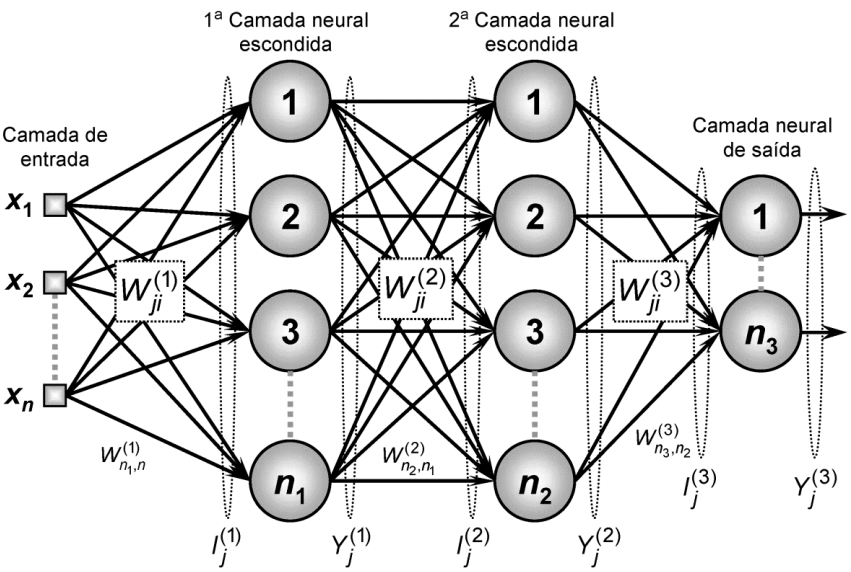
$$Y_j^{(3)} = g_3\left(I_j^{(3)}\right) = \left[g_3\left(I_1^{(3)}\right)\right] = [\tanh(0.76)] = [0.64]$$

Perceptron Multi-Camadas: Backpropagation



- Normalmente o algoritmo pode ser dividido em 2 partes:
  - Ajuste da camada de saída
  - Ajuste das demais camadas
- Os ajustes do Backpropagation são baseados na **Regra Delta do ADALINE**, ou seja, através do uso do Gradiente Descendente do Erro.

Perceptron Multi-Camadas: Backpropagation



- **<u>Função de Custo (Erro)</u>**:
  - Considerando a k-ésima amostra de treinamento para a topologia ilustrada abaixo, assume-se a função erro quadrático como aquela a ser utilizada para medir o desempenho local associado aos resultados produzidos pelos neurônios de saída frente à referida amostra:

$$E(k) = \frac{1}{2}\sum_{j=1}^{n_3}\left(d_j(k) - Y_j^{(3)}(k)\right)^2$$

  - para um conjunto de treinamento composto por p amostras, a evolução do desempenho global do aprendizado pode ser feito por meio da avaliação do “erro quadrático médio”:

$$E_M = \frac{1}{p}\sum_{k=1}^{p} E(k)$$

Perceptron Multi-Camadas: Backpropagation

- **Ajuste da Camada de Saída**: ajustar a matriz $W_{ji}^{(3)}$ a fim de minimizar o erro entre a saída da rede frente à saída desejada.

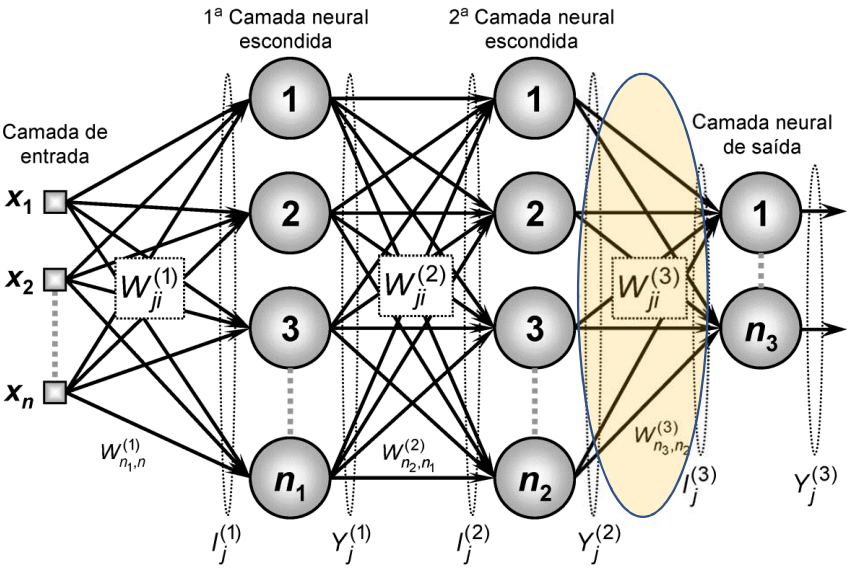
$$I_j^{(3)} = W_{ji}^{(3)} \cdot Y_i^{(2)} \quad \rightarrow \quad Y_j^{(3)} = g_3\left(I_j^{(3)}\right) \quad \rightarrow \quad E = \frac{1}{2}\left(d_j - Y_j^{(3)}\right)^2$$

$$\frac{\partial I_j^{(3)}}{\partial W_{ji}^{(3)}} = Y_i^{(2)} \quad \leftarrow \quad \frac{\partial Y_j^{(3)}}{\partial I_j^{(3)}} = g_3{}'\left(I_j^{(3)}\right) \quad \leftarrow \quad \frac{\partial E}{\partial Y_j^{(3)}} = -\left(d_j - Y_j^{(3)}\right)$$

$$\nabla E^{(3)} = \frac{\partial I_j^{(3)}}{\partial W_{ji}^{(3)}} \cdot \frac{\partial Y_j^{(3)}}{\partial I_j^{(3)}} \cdot \frac{\partial E}{\partial Y_j^{(3)}} = Y_i^{(2)} \cdot g_3'\left(I_j^{(3)}\right) \cdot \left[-\left(d_j - Y_j^{(3)}\right)\right]$$

$$-\delta_j^{(3)} \quad \rightarrow \quad \delta_j^{(3)} = g_3'\left(I_j^{(3)}\right) \cdot \left(d_j - Y_j^{(3)}\right)$$

Perceptron Multi-Camadas: Backpropagation



- **<u>Ajuste da Camada de Saída</u>**: ocorre na direção oposta ao gradiente:

$$\Delta W_{ji}^{(3)} = -\eta \cdot \nabla E^{(3)}$$

$$W_{ji}^{(3)atual} = W_{ji}^{(3)anterior} + \eta . \delta_j^{(3)} . Y_i^{(2)}$$

$$\delta_j^{(3)} = \left(d_j - Y_j^{(3)}\right) . g'(I_j^{(3)})$$

- Onde $\delta_j^{(3)}$ representa o gradiente local em relação ao j-ésimo neurônio da camada de saída.

Perceptron Multi-Camadas: Backpropagation

- **Ajuste da 2ª Camada Oculta**: ajustar a matriz $W_{ji}^{(2)}$ a fim de minimizar o erro. Aqui não se dispõe do erro gerado na saída da rede, então é utilizado o valor "propagado" por este erro no cálculo da correção, ou seja, a contribuição do erro devido a todas as sinapses que partem da camada intermediária que está sendo corrigida.

$$I_j^{(2)} = W_{ji}^{(2)} \cdot Y_i^{(1)} \rightarrow Y_j^{(2)} = g_2\left(I_j^{(2)}\right) \rightarrow I_j^{(3)} = W_{ji}^{(3)} \cdot Y_i^{(2)} \dashrightarrow E\left(Y_j^{(2)}\right)$$

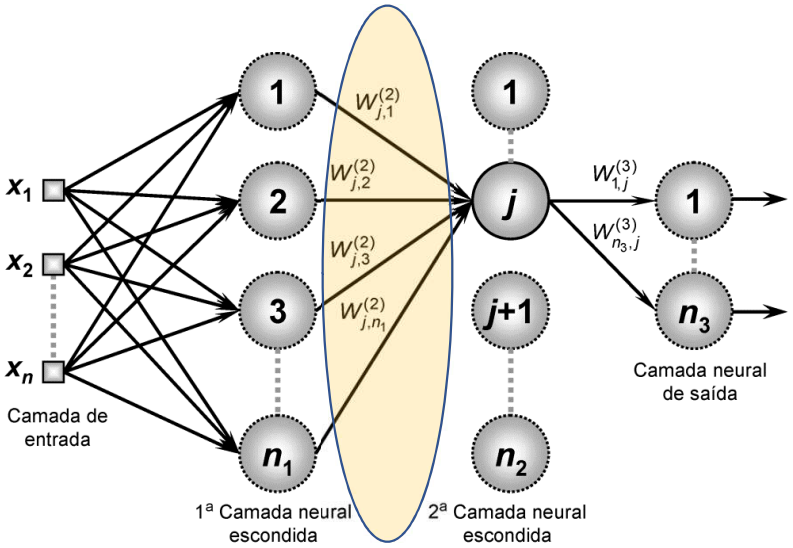
$$\frac{\partial I_j^{(2)}}{\partial W_{ji}^{(2)}} = Y_i^{(1)} \leftarrow \frac{\partial Y_j^{(2)}}{\partial I_j^{(2)}} = g_2{}'\left(I_j^{(2)}\right) \leftarrow \frac{\partial E}{\partial Y_j^{(2)}} = \sum_{k=1}^{n_3} \frac{\partial E}{\partial I_k^{(3)}} \cdot \frac{\partial I_k^{(3)}}{\partial Y_j^{(2)}}$$

$$\frac{\partial I_k^{(3)}}{\partial Y_j^{(2)}} = W_{kj}^{(3)} \qquad \frac{\partial E}{\partial I_k^{(3)}} = -\delta_k^{(3)}$$

$$\frac{\partial E}{\partial W_{ji}^{(2)}} = \frac{\partial I_j^{(2)}}{\partial W_{ji}^{(2)}} \cdot \frac{\partial Y_j^{(2)}}{\partial I_j^{(2)}} \cdot \frac{\partial E}{\partial Y_j^{(2)}} = Y_i^{(1)} \cdot g_2'\left(I_j^{(2)}\right) \cdot \left(-\sum_{k=1}^{n_3} W_{kj}^{(3)} \cdot \delta_k^{(3)}\right)$$

$$-\delta_j^{(2)} \dashrightarrow \delta_j^{(2)} = g_2'\left(I_j^{(2)}\right) \cdot \sum_{k=1}^{n_3} W_{kj}^{(3)} \cdot \delta_k^{(3)}$$

Perceptron Multi-Camadas: Backpropagation



- **<u>Ajuste da Camada Oculta</u>**: ajustar a matriz $W_{ji}^{(2)}$ a fim de minimizar o erro.

  Aqui não se dispõe do erro gerado na saída da rede, então é utilizado o valor "propagado" por este erro no cálculo da correção, ou seja, a contribuição do erro devido a todas as sinapses que partem da camada intermediária que está sendo corrigida:

$$W_{ji}^{(2)atual} = W_{ji}^{(2)anterior} + \eta . \delta_j^{(2)} . Y_i^{(1)}$$

$$\delta_j^{(2)} = -\left(\sum_{k=1}^{n3} W_{kj}^{(3)} \cdot \delta_k^{(3)} \right) . g'(I_j^{(2)})$$

- Onde $\delta_j^{(2)}$ representa o gradiente local.

Perceptron Multi-Camadas: Backpropagation

- **Ajuste da 1ª Camada Oculta**: ajustar a matriz $W_{ji}^{(1)}$ a fim de minimizar o erro. Aqui não se dispõe do erro gerado na saída da rede, então é utilizado o valor "propagado" por este erro no cálculo da correção, ou seja, a contribuição do erro devido a todas as sinapses que partem da camada intermediária que está sendo corrigida.

$$I_j^{(1)} = W_{ji}^{(1)} \cdot Y_i^{(0)} \rightarrow Y_j^{(1)} = g_2\left(I_j^{(1)}\right) \rightarrow I_j^{(2)} = W_{ji}^{(2)} \cdot Y_i^{(1)} \dashrightarrow E\left(Y_j^{(1)}\right)$$

$$\frac{\partial I_j^{(1)}}{\partial W_{ji}^{(1)}} = Y_i^{(0)} \leftarrow \frac{\partial Y_j^{(1)}}{\partial I_j^{(1)}} = g_1{}'\left(I_j^{(1)}\right) \leftarrow \frac{\partial E}{\partial Y_j^{(1)}} = \sum_{k=1}^{n_2} \frac{\partial E}{\partial I_k^{(2)}} \cdot \frac{\partial I_k^{(2)}}{\partial Y_j^{(1)}}$$

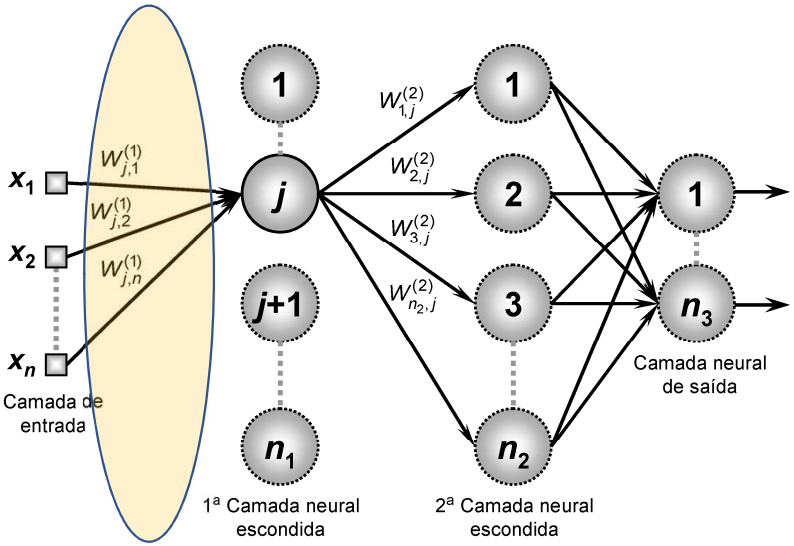
$$\frac{\partial I_k^{(2)}}{\partial Y_j^{(1)}} = W_{kj}^{(2)} \qquad \frac{\partial E}{\partial I_k^{(2)}} = -\delta_k^{(2)}$$

$$Y_i^{(0)} = x_i$$

$$\frac{\partial E}{\partial W_{ji}^{(1)}} = \frac{\partial I_j^{(1)}}{\partial W_{ji}^{(1)}} \cdot \frac{\partial Y_j^{(1)}}{\partial I_j^{(1)}} \cdot \frac{\partial E}{\partial Y_j^{(1)}} = Y_i^{(0)} \cdot g_1'\left(I_j^{(1)}\right) \cdot \left(-\sum_{k=1}^{n_2} W_{kj}^{(2)} \cdot \delta_k^{(2)}\right)$$

$$-\delta_j^{(1)} \dashrightarrow \delta_j^{(1)} = g_1'\left(I_j^{(1)}\right) \cdot \sum_{k=1}^{n_2} W_{kj}^{(2)} \cdot \delta_k^{(2)}$$

Perceptron Multi-Camadas: Backpropagation



$$Y_i^{(0)} = x_i$$

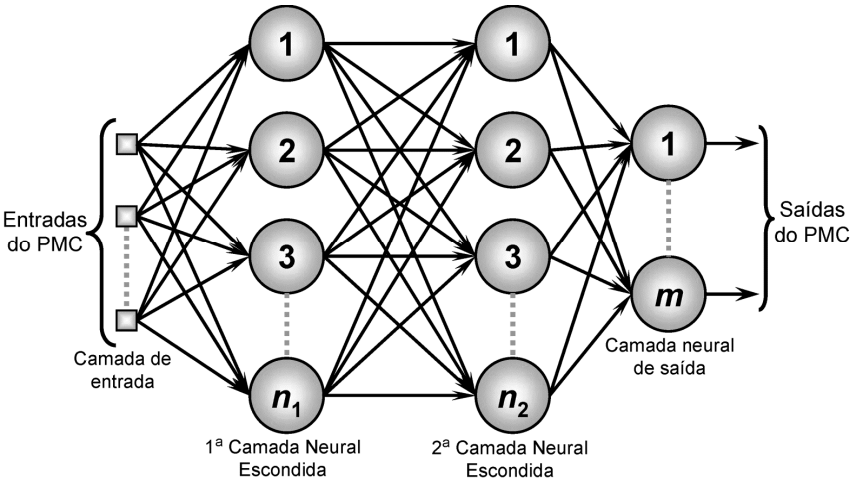
- **Ajuste da Camada de Entrada**: ajustar a matriz $W_{ji}^{(1)}$ a fim de minimizar o erro. Aqui não se dispõe do erro gerado na saída da rede, então é utilizado o valor "propagado" por este erro no cálculo da correção, ou seja, a contribuição do erro devido a todas as sinapses que partem da camada intermediária que está sendo corrigida:

$$W_{ji}^{(1)atual} = W_{ji}^{(1)anterior} + \eta . \delta_j^{(1)} . Y_i^{(0)}$$

$$\delta_j^{(1)} = -\left(\sum_{k=1}^{n2} W_{kj}^{(2)} \cdot \delta_k^{(2)} \right) . g'(I_j^{(1)})$$

- Onde $\delta_j^{(1)}$ representa o gradiente local.

Perceptron Multi-Camadas : Treinamento



Início {Algoritmo PMC – Fase de Treinamento}

<1> Obter o conjunto de amostras de treinamento $\{\boldsymbol{x}^{(k)}\}$;

<2> Associar o vetor de saída desejada $\{\boldsymbol{d}^{(k)}\}$ para cada amostra;

<3> Iniciar $w_{ji}^{(1)}$, $w_{ji}^{(2)}$ e $w_{ji}^{(3)}$ com valores aleatórios pequenos;

<4> Especificar taxa de aprendizagem $\{\eta\}$ e precisão requerida $\{\varepsilon\}$;

<5> Iniciar o contador de número de épocas $\{época \leftarrow 0\}$;

<6> Repetir as instruções:

<6.1> $E_M^{anterior} \leftarrow E_M$; {conforme (5.8)}

<6.2> Para todas as amostras de treinamento $\{\boldsymbol{x}^{(k)}, \boldsymbol{d}^{(k)}\}$, fazer:

<6.2.1> Obter $I_j^{(1)}$ e $Y_j^{(1)}$; {conforme (5.1) e (5.4)}

<6.2.2> Obter $I_j^{(2)}$ e $Y_j^{(2)}$; {conforme (5.2) e (5.5)}

<6.2.3> Obter $I_j^{(3)}$ e $Y_j^{(3)}$; {conforme (5.3) e (5.6)}

} Passo *Forward*

<6.2.4> Determinar $\delta_j^{(3)}$; {conforme (5.15)}

<6.2.5> Ajustar $w_{ji}^{(3)}$; {conforme (5.17)}

<6.2.6> Determinar $\delta_j^{(2)}$; {conforme (5.26)}

<6.2.7> Ajustar $w_{ji}^{(2)}$; {conforme (5.28)}

<6.2.8> Determinar $\delta_j^{(1)}$; {conforme (5.37)}

<6.2.9> Ajustar $w_{ji}^{(1)}$; {conforme (5.39)}

} Passo *Backward*

<6.3> Obter $Y_j^{(3)}$ ajustado; {conforme <6.2.1>, <6.2.2> e <6.2.3>}

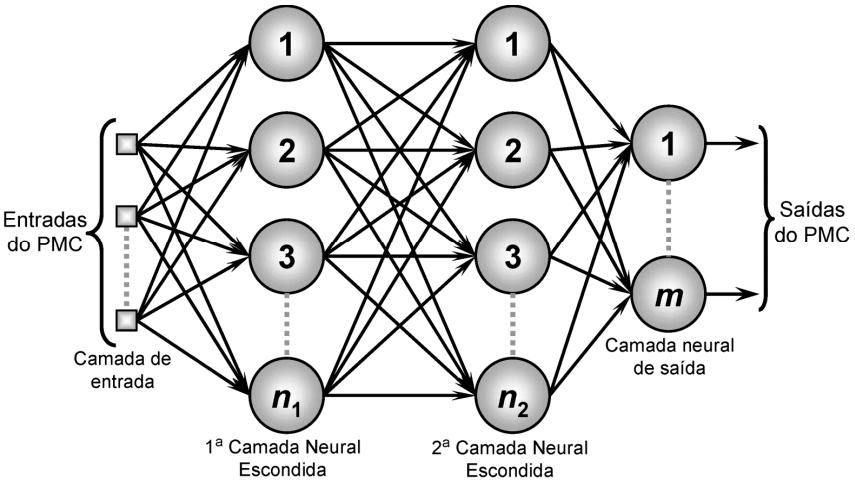
<6.4> $E_M^{atual} \leftarrow E_M$; {conforme (5.8)}

<6.5> $época \leftarrow época + 1$;

Até que: $|E_M^{atual} - E_M^{anterior}| \leq \varepsilon$

Fim {Algoritmo PMC – Fase de Treinamento}

Perceptron Multi-Camadas : Operação



Início {Algoritmo PMC – Fase de Operação}

<1> Obter uma amostra { ***x*** };

<2> Assumir $W_{ji}^{(1)}$, $W_{ji}^{(2)}$ e $W_{ji}^{(3)}$ já ajustadas no treinamento;

<3> Execute as seguintes instruções:

- <3.1> Obter $I_j^{(1)}$ e $Y_j^{(1)}$; {conforme (5.1) e (5.4)}
- <3.2> Obter $I_j^{(2)}$ e $Y_j^{(2)}$; {conforme (5.2) e (5.5)}
- <3.3> Obter $I_j^{(3)}$ e $Y_j^{(3)}$; {conforme (5.3) e (5.6)}

} Passo *Forward*

<4> Disponibilizar as saídas da rede, as quais são dadas pelos elementos contidos em $Y_j^{(3)}$

Fim {Algoritmo PMC – Fase de Operação}

Não esqueça do -1 no vetor de entradas: $x^{(k)} = \left[-1, x_1^{(k)}, x_2^{(k)} \ldots x_n^{(k)}\right]^T$



PMC: Aspectos Complementares dos Dados

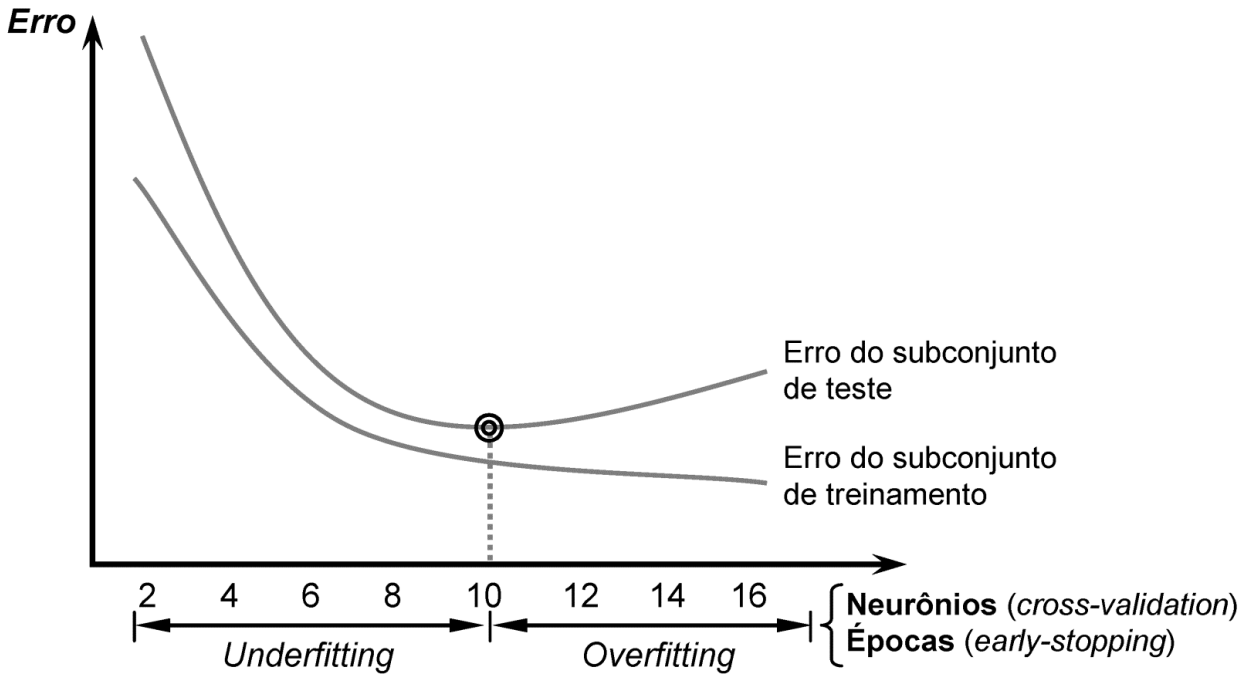
Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Perceptron Multi-Camadas : Validação Cruzada

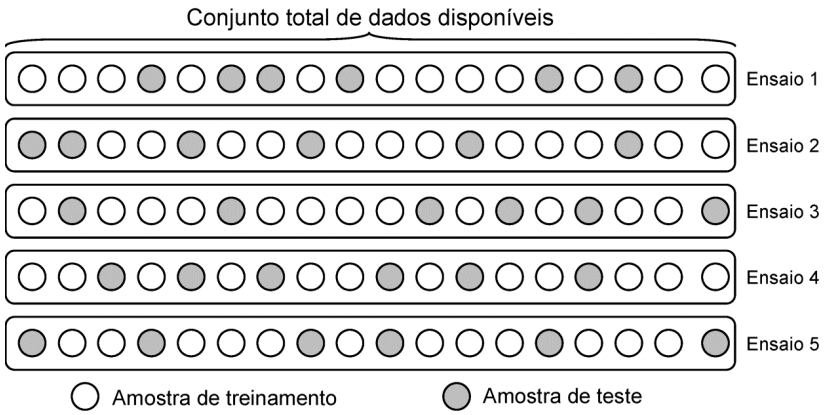


- A especificação da topologia de rede PMC mais apropriada para mapear um problema específico é usualmente efetuada de forma empírica.
- Como exemplo ilustrativo, considera-se que para um determinado problema se tem 4 topologias candidatas de PMC:
  - Topologia Candidata 1: 05 neurônios na camada escondida.
  - Topologia Candidata 2: 10 neurônios na camada escondida.
  - Topologia Candidata 3: 15 neurônios na camada escondida.
  - Topologia Candidata 4: 20 neurônios na camada escondida.
- Objetivo: definir qual delas seria a mais indicada para executar o mapeamento do referido problema.

Perceptron Multi-Camadas : Validação Cruzada



Perceptron Multi-Camadas : Validação Cruzada

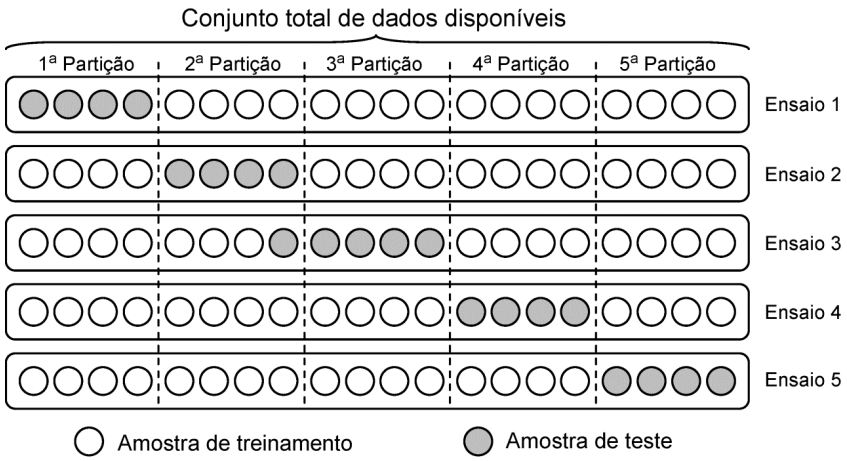


- **Amostragem aleatória**: o conjunto total de dados (amostras) disponíveis é aleatoriamente dividido em duas partes, isto é, subconjunto de treinamento e subconjunto de teste (validação).
  - **Subconjunto de treinamento**: utilizado para treinar todas as topologias candidatas.
  - **Subconjunto de teste**: utilizado para selecionar aquela que estará apresentando os melhores resultados de generalização.

É comum usar 1/3 das amostras para testes, deixando os 2/3 restantes para o treinamento da rede.

É recomendável repetir esta técnica algumas vezes, com conjuntos diferentes a fim de garantir que os dados usados sejam representativos do problema;

Perceptron Multi-Camadas : Validação Cruzada

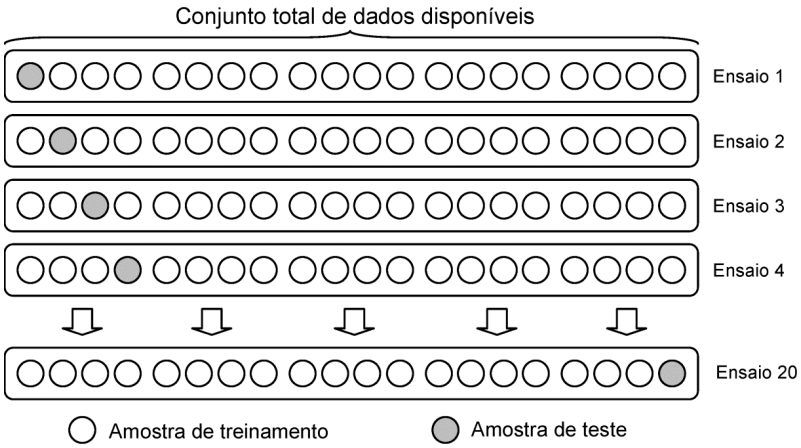


Conjunto total de amostras → 20
Valor do parâmetro k → 5

O desempenho global de cada topologia candidata será agora também obtido em função da média entre os desempenhos individuais observados quando da aplicação das k-partições

- **k-partições**: realiza-se aqui a divisão do conjunto total de amostras em k partições, sendo que (k-1) delas serão usadas para compor o subconjunto de treinamento, ao passo que a partição restante constituirá o subconjunto de teste
  - Por conseguinte, o processo de aprendizado se repete k vezes até que todas as partições tenham sido utilizadas como subconjunto de teste.

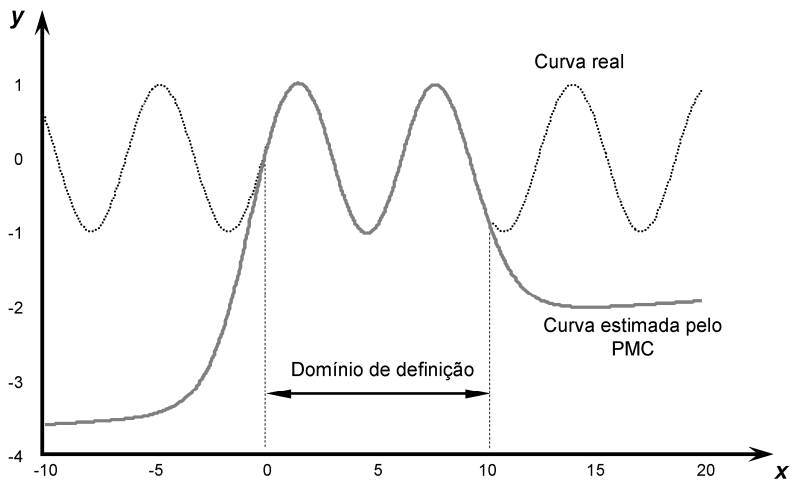
Perceptron Multi-Camadas : Validação Cruzada



- **por unidade (leave one out)**: consiste da utilização de uma única amostra para o subconjunto de teste, sendo todas as demais alocadas para o subconjunto de treinamento.
  - O processo de aprendizado é então repetido até que todas as amostras sejam individualmente utilizadas como subconjunto de teste.

Possui elevado esforço computacional, pois o processo de aprendizagem será repetido, considerando cada uma das topologias candidatas, um número de vezes que será igual ao tamanho do conjunto total de amostras.

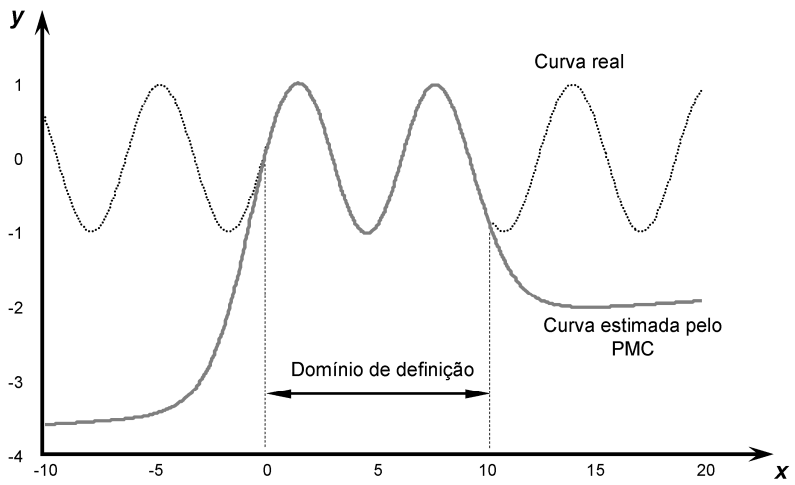
Perceptron Multi-Camadas : Relevância dos dados



- Assegurar que todas as amostras, que carregam os valores mínimos e máximos de cada variável de entrada, estejam também dentro desses subconjuntos.

- Caso contrário, se tais valores forem inadvertidamente alocados aos subconjuntos de teste, o PMC poderia então gerar erros significativos, pois tentaria generalizar valores que estão fora dos domínios de definição de suas variáveis de entrada (nos quais foi treinado).

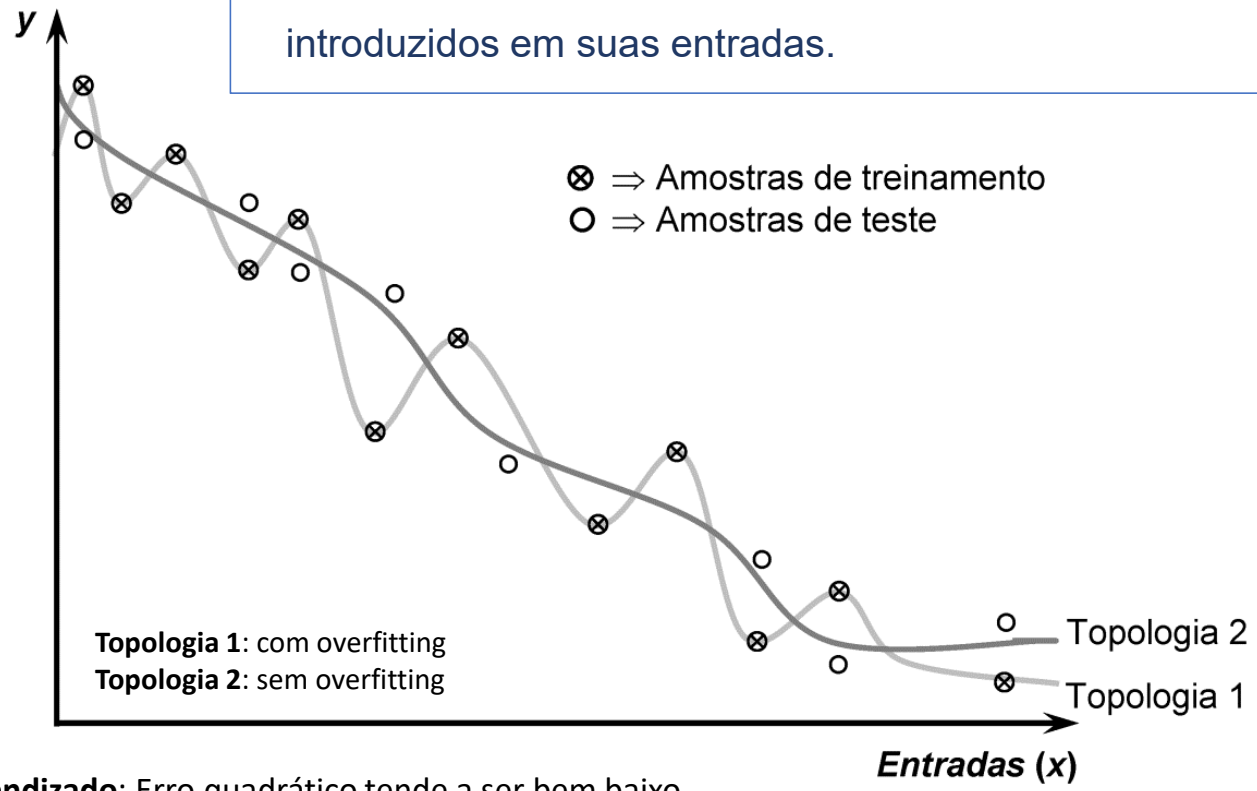
Perceptron Multi-Camadas : Relevância dos dados



- Durante toda a fase de operação, deve-se ainda garantir que os atuais sinais, referentes a cada uma das variáveis de entrada, estejam novamente compreendidos dentro daqueles domínios de definição que foram obtidos a partir dos valores mínimos e máximos dos subconjuntos de treinamento.

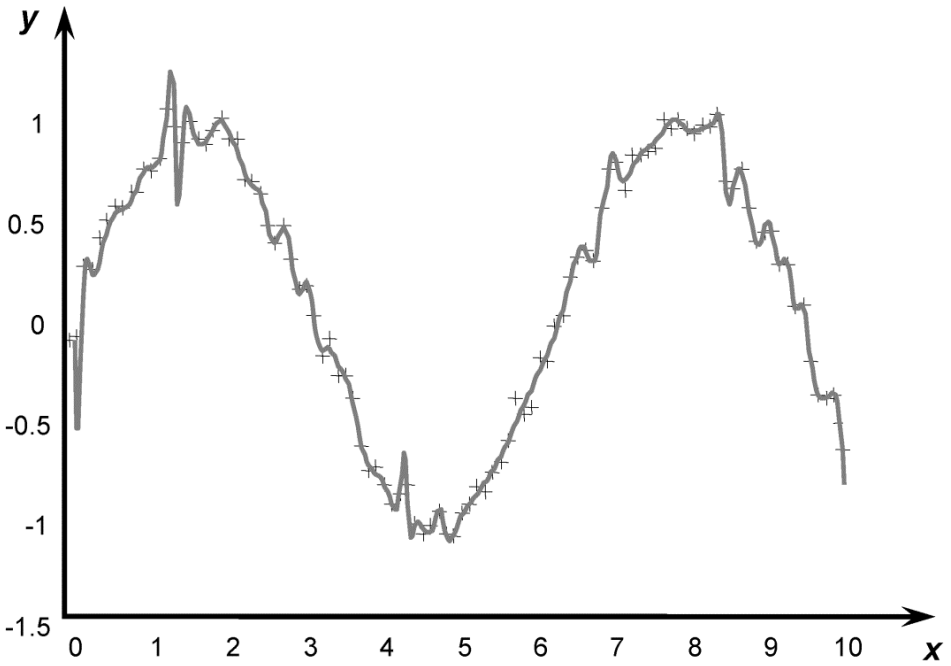
PMC: Overfitting e Underfitting

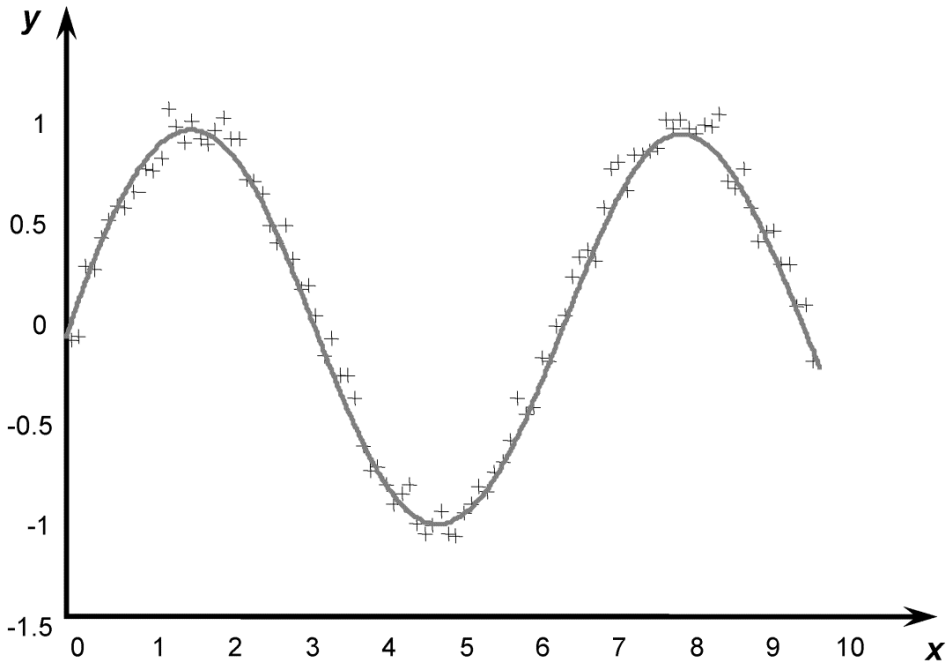
- **overfitting**: memorização excessiva, em que o PMC acaba decorando as suas respostas frente aos estímulos introduzidos em suas entradas.



Durante a fase de aprendizado: Erro quadrático tende a ser bem baixo.

Durante a fase de teste (generalização): Erro quadrático já tende a assumir valores bem elevados frente às amostras do subconjunto de teste.





PMC: Overfitting e Underfitting

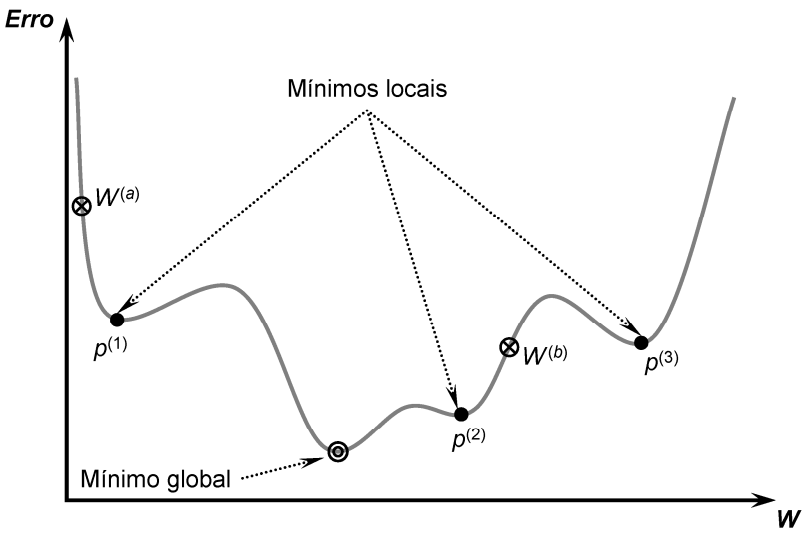
Perceptron Multi-Camadas : Overfitting e Underfitting



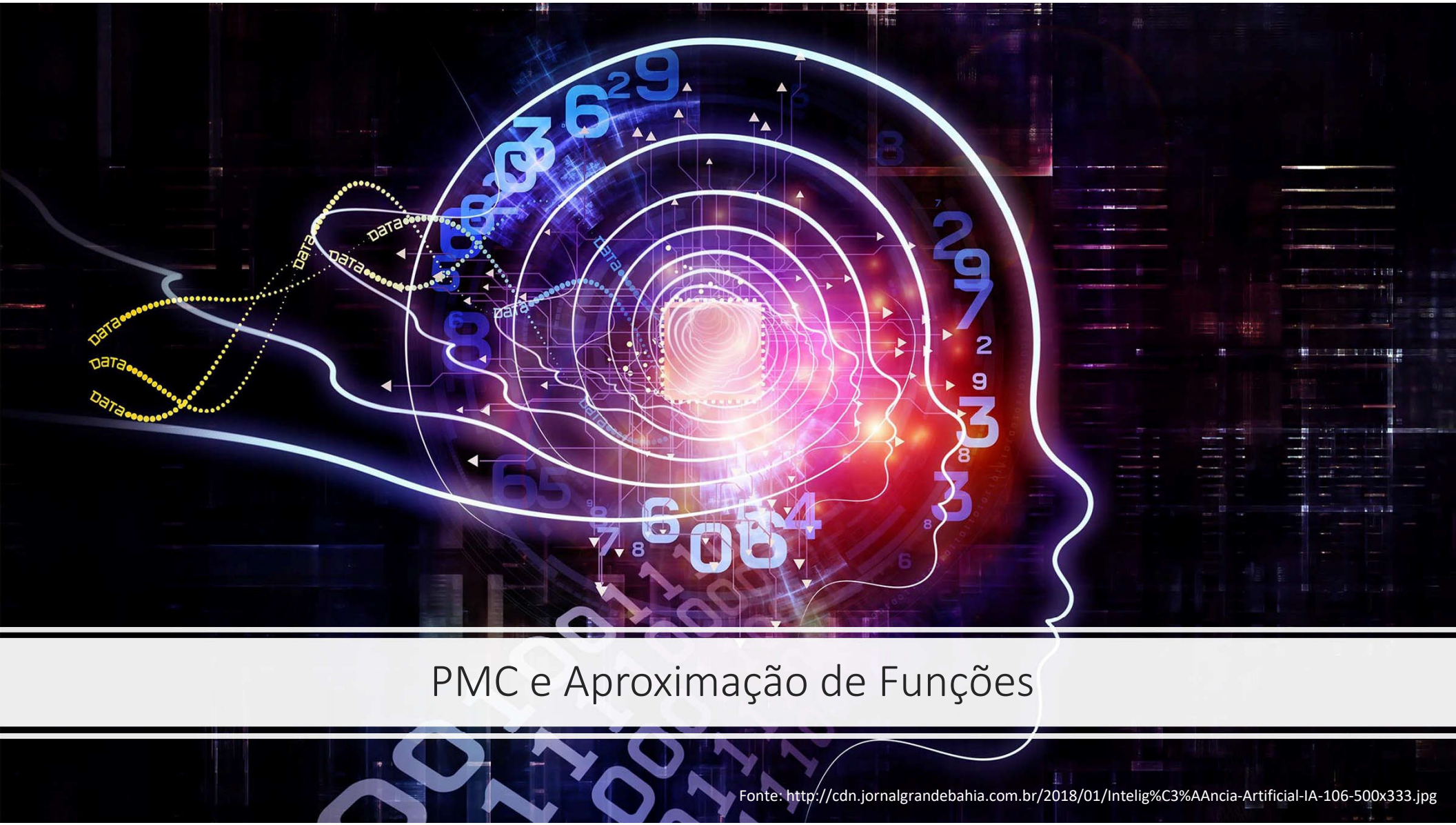
- **Underfitting**: uma topologia de PMC com número muito reduzido de neurônios pode ser insuficiente para a extração e armazenamento de características que permitam à rede implementar as hipóteses a respeito do comportamento do processo.

- Nesses casos o erro quadrático tanto na fase de aprendizado como na fase de teste serão bem significativos.

Perceptron Multi-Camadas : Superfície de Erro e mínimos locais



- Como a superfície de erro produzida pelo PMC é não-linear, há a possibilidade de que o treinamento leve a matriz de pesos da rede p/ um ponto de mínimo local.
- Este ponto pode não corresponder aos valores mais apropriados aos propósitos de generalização de resultados.
- Esta tendência de convergência fica condicionada à posição em que W foi iniciada, pois o treinamento das redes é baseado em métodos de gradiente descendente.



PMC e Aproximação de Funções

Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Perceptron Multi-Camadas : Aproximação de Funções



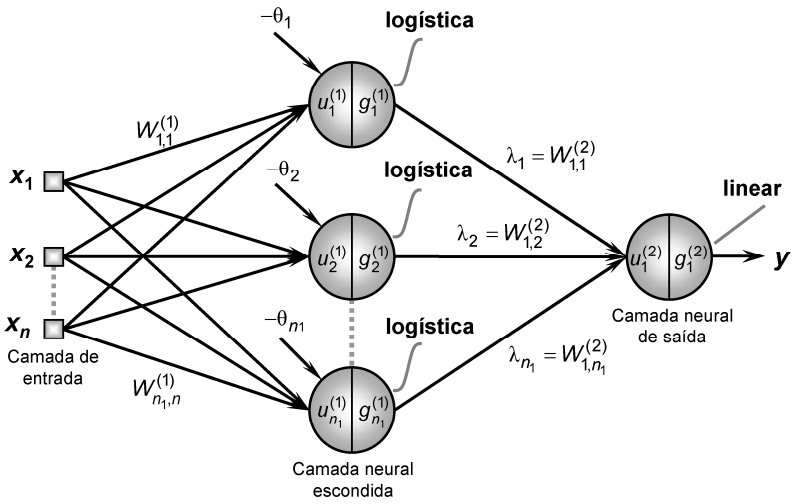
- É a classe de problemas em que as redes PMC podem usufruir de maior destaque.
- Consiste de mapear o comportamento de um processo se baseando somente em diversas medições efetivadas em suas entradas e saídas (sem conhecer a modelagem matemática).
- Observa-se aqui uma das principais características intrínsecas das redes neurais artificiais, ou seja, o aprendizado a partir de exemplos.
- No caso de aproximação de funções, traduz-se na disponibilização de um conjunto de entradas/saídas que reproduzem o comportamento do sistema a ser tratado.

Perceptron Multi-Camadas : Aproximação de Funções



- Nesse contexto, constata-se que as RNA têm sido extensivamente aplicados nas seguintes situações:
  - O processo a ser modelado é de certa forma complexo.
  - Naqueles casos em que as utilizações de métodos convencionais produzem resultados insatisfatórios.
  - Naqueles casos em que os sistemas convencionais exigem requisitos computacionais bem sofisticados

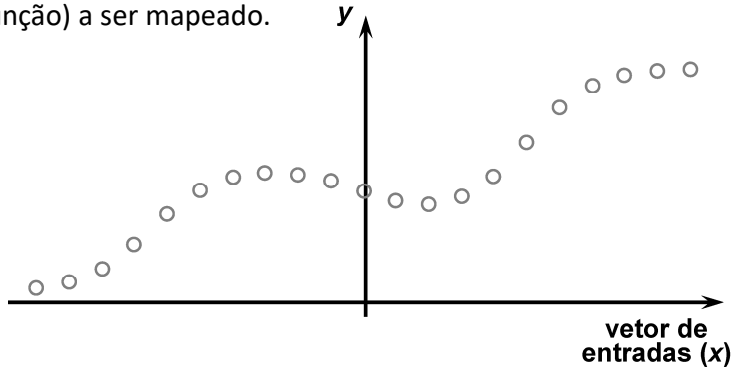
Perceptron Multi-Camadas : Aproximação de Funções

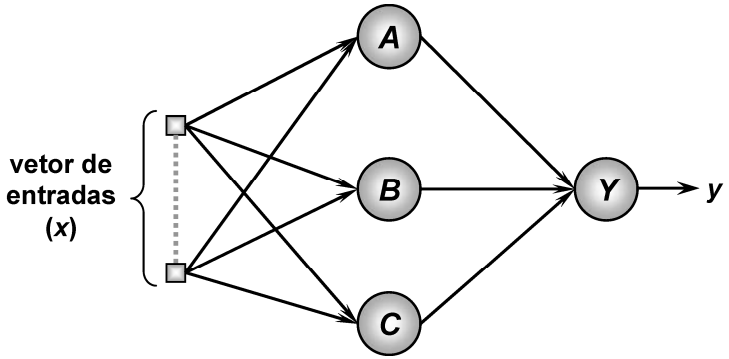


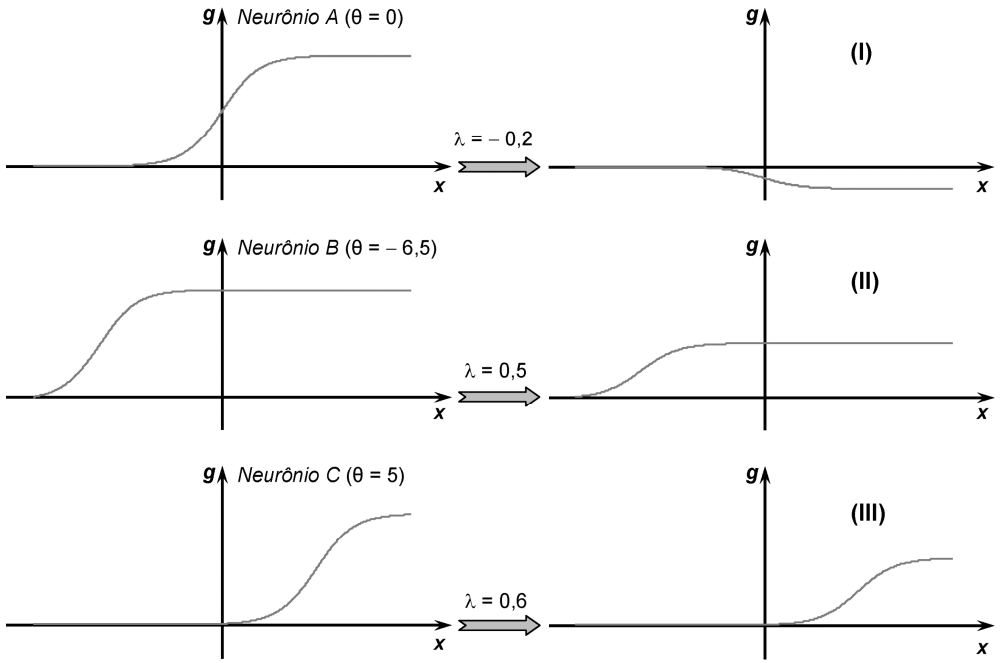
- **<u>Teorema da Aproximação Universal</u>**: Baseado nas demonstrações de Kolmogorov, estas fornecem as bases para se definir as configurações de redes PMC p/ finalidade de mapear funções algébricas:
  - Assumindo que g(.) a ser adotada nas redes PMC sejam contínuas e limitadas em suas imagens, tais como a logística e tangente hiperbólica, demonstra-se então que:
  - Um PMC, composto de apenas uma camada escondida, é capaz de mapear qualquer função contínua no espaço real. Em termos matemáticos, tem-se que **a função de ativação da camada de saída será constituída por uma função linear**.

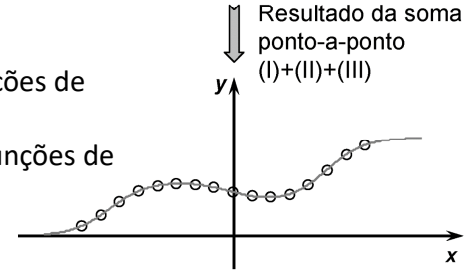
Perceptron Multi-Camadas : Aproximação de Funções

Conjunto de amostras relacionando entradas/saídas referente ao processo (função) a ser mapeado.









Configuração de PMC após o ajuste de seus pesos:

- Parâmetro θ: responsável pela translação das funções de ativação.
- Parâmetro λ: responsável pelo escalamento das funções de ativação



PMC e Classificação de Padrões

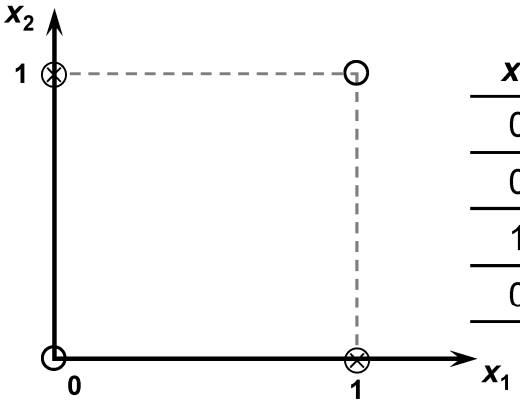
Fonte: http://cdn.jornalgrandebahia.com.br/2018/01/Intelig%C3%AAncia-Artificial-IA-106-500x333.jpg

Perceptron Multi-Camadas : Classificação de Padrões

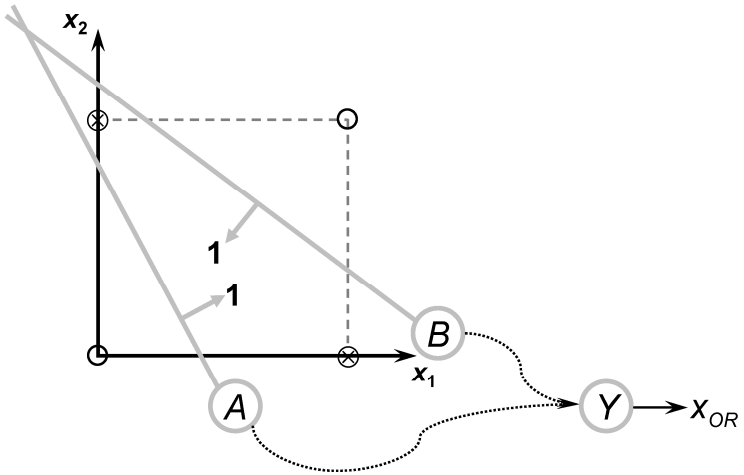


- Redes PMC podem ser consideradas as mais utilizadas na solução de problemas advindos das mais variadas áreas de conhecimento.
- Redes PMC são as mais amplamente empregadas em diferentes temáticas envolvendo as engenharias como um todo, em especial a Engenharia Elétrica.
- O problema de **Classificação de Padrões** consiste de associar um padrão de entrada (amostra) para uma daquelas classes que já foram previamente definidas.
- Diferentemente dos problemas envolvendo Aproximação Funcional (saídas reais/analógicas), as respostas associadas aos problemas de Classificação de Padrões estão sempre relacionadas com grandezas discretas (enumeráveis)

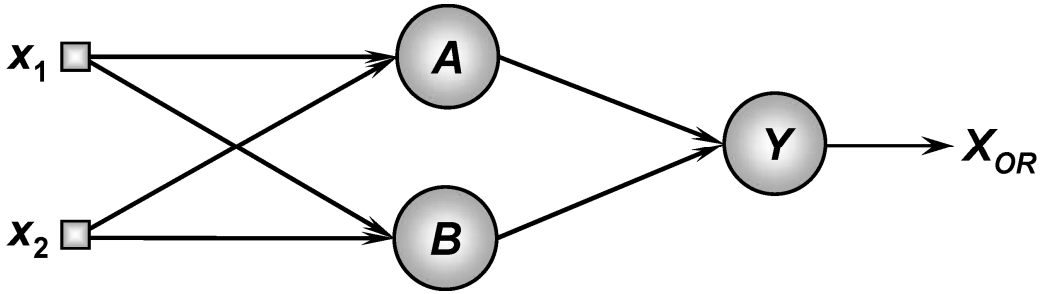
Perceptron Multi-Camadas : Classificação de Padrões



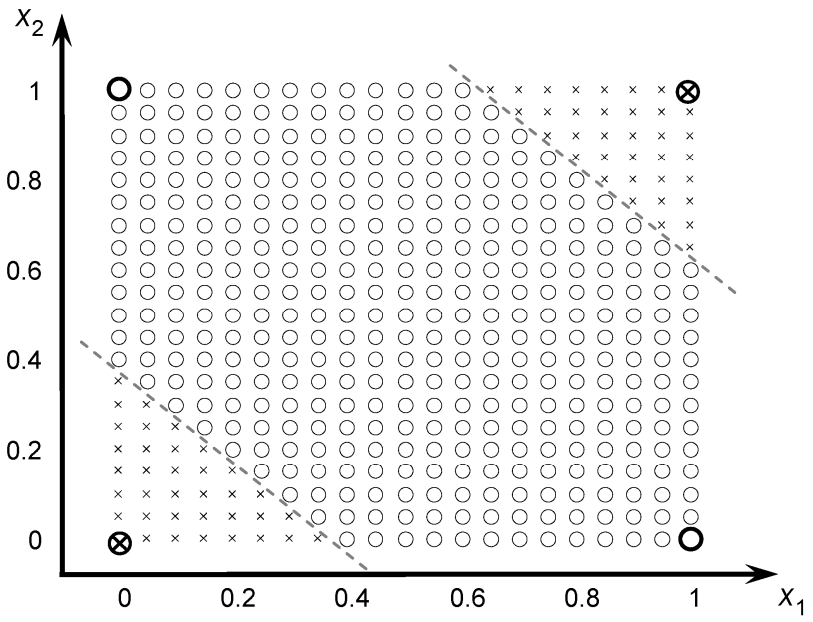
x_1	x_2	X_{OR}
0	0	$0 \rightarrow O$
0	1	$1 \rightarrow \otimes$
1	0	$1 \rightarrow \otimes$
0	1	$0 \rightarrow O$



- **Problema do OU-exclusivo (XOR)**: o Perceptron Simples não consegue separar e classificar os padrões.
- Topologia de PMC para resolver o problema do ou-exclusivo. Nota-se aqui que o PMC tem apenas uma Camada Oculta.



Perceptron Multi-Camadas : Classificação de Padrões

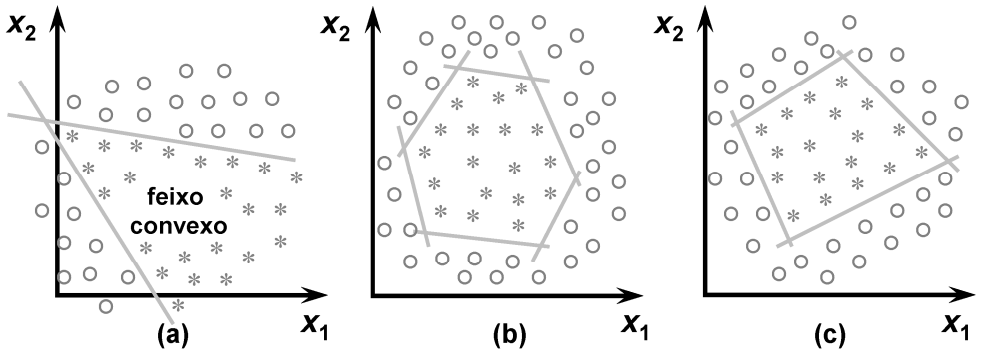


- A figura mostra as fronteiras de classificação associadas ao problema do ou-exclusivo.
- As fronteiras são delimitadas por duas retas, as quais são formadas quando se executa a interseção da superfície ilustrada acima (Saída do neurônio Y) próximo ao plano y = 0.

Perceptron Multi-Camadas : Classificação de Padrões



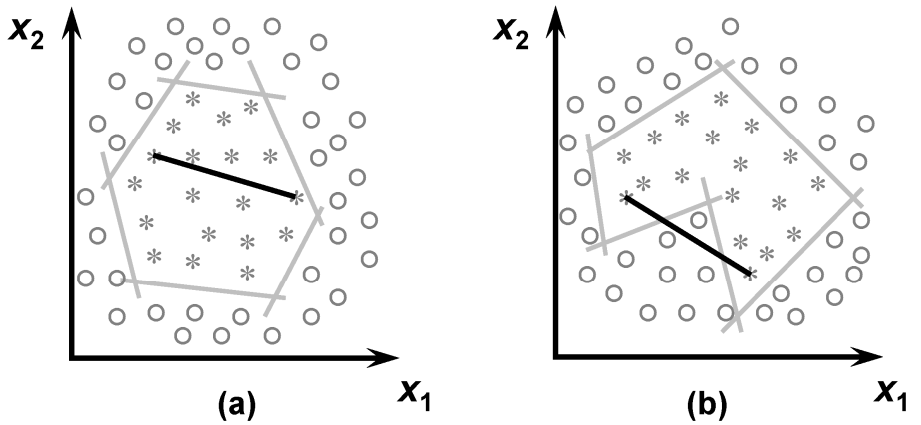
- De forma similar ao problema do ou-exclusivo, pode-se então deduzir que um PMC de Uma Camada Escondida (duas camadas neurais) é capaz de mapear qualquer problema de classificação de padrões cujos elementos estejam dentro de uma Região Convexa.



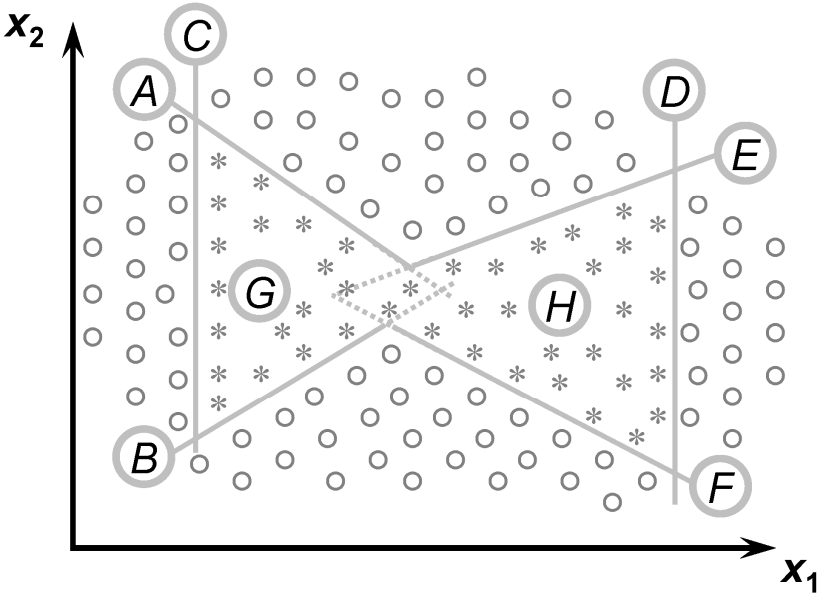
Perceptron Multi-Camadas : Classificação de Padrões



- Do ponto de vista geométrico, uma região é considerada Convexa se, e somente se, todos os pontos contidos em quaisquer segmentos de reta, os quais estão também definidos por quaisquer dois pontos delimitados pelo respectivo domínio, estiverem ainda dentro da mesma.

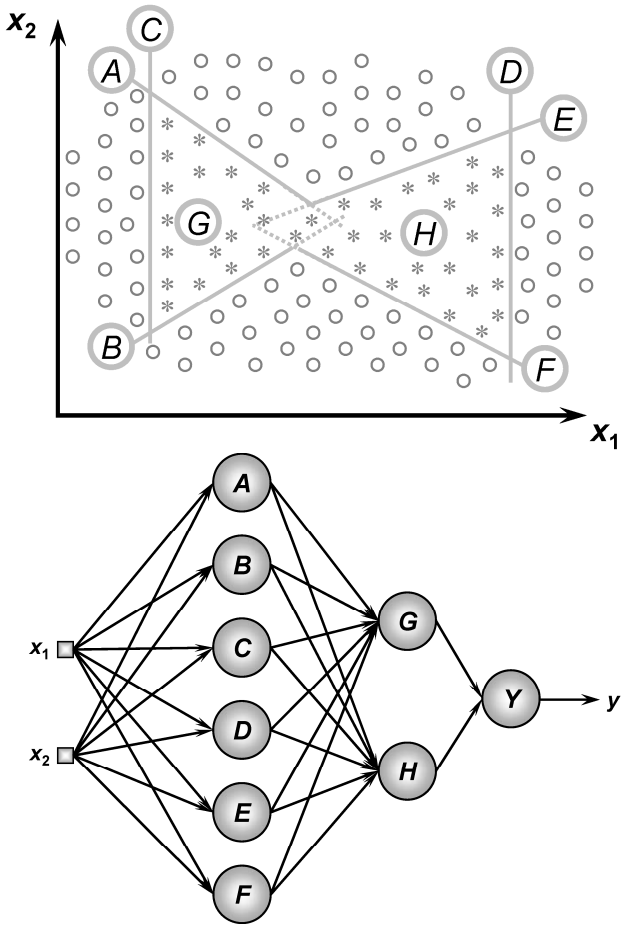


Perceptron Multi-Camadas : Classificação de Padrões



- Do slide anterior, conclui-se que PMC de Uma Camada Escondida consegue classificar padrões dispostos em Regiões Convexas.
- Conseqüentemente, pode-se também deduzir que redes PMC de Duas Camadas Escondidas são capazes de classificar padrões que estejam em quaisquer tipos de regiões geométricas.

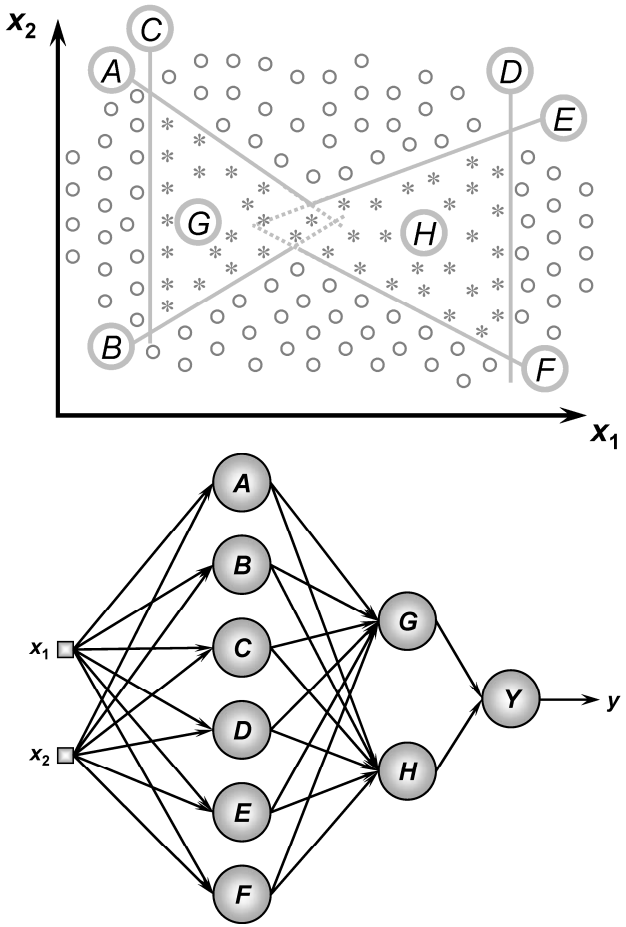
Perceptron Multi-Camadas : Classificação de Padrões



- **Neurônios da 1a Camada Escondida**:
  - Os neurônios A, B e C são responsáveis pela delimitação da 1ª RC (triângulo da esquerda).
  - Os neurônios D, E e F são responsáveis pela delimitação da 2ª RC (triângulo da direita).

- **Neurônios da 2a Camada Escondida:**
  - Neurônio G combina saídas dos neurônios A, B e C, a fim de enquadrar a parcela das amostras que pertenceriam à 1ª RC, sendo que nesta condição a sua saída seria igual a 1.
  - Neurônio H combina saídas dos neurônios D, E e F, a fim de enquadrar a 2ª RC, produzindo então valor 1.

Perceptron Multi-Camadas : Classificação de Padrões

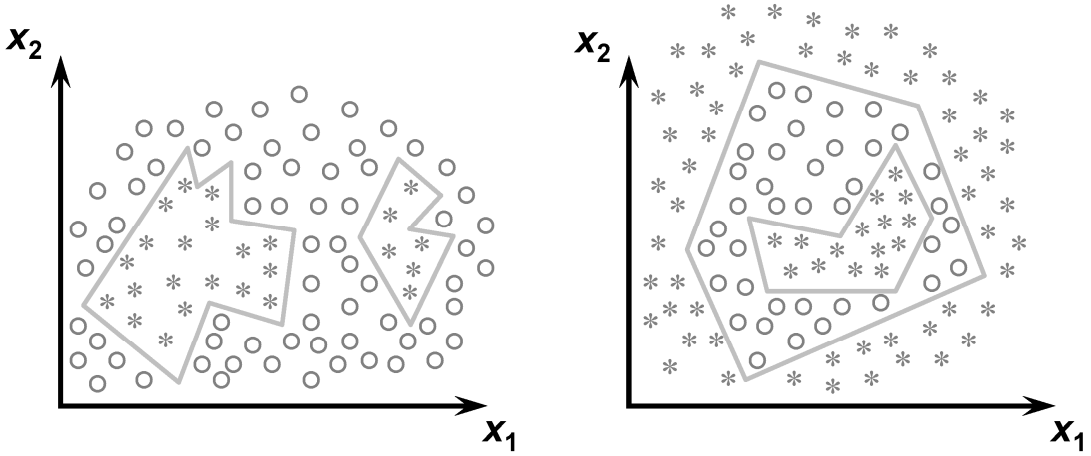


- **Neurônios da Camada de Saída**:
  - Neurônio Y estaria então incumbido de efetuar uma operação de disjunção booleana (porta OR), pois se qualquer uma das saídas produzidas pelos neurônios G ou H for igual a 1, sua resposta final y seria também igual a 1.

Perceptron Multi-Camadas : Classificação de Padrões

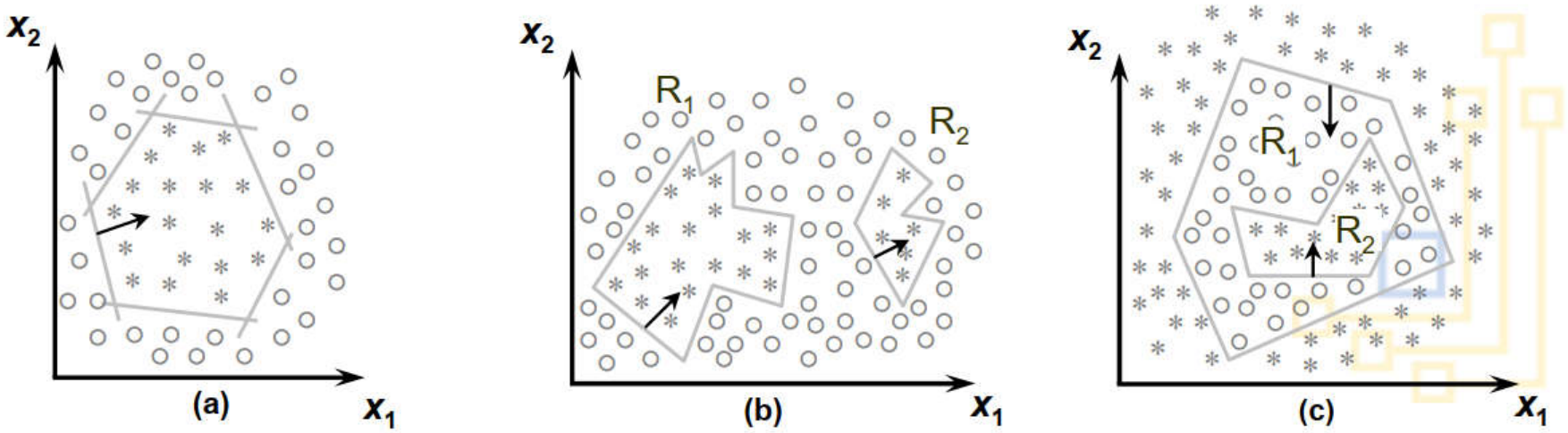


- Adicionalmente, o PMC com duas camadas escondidas poderia ainda mapear outros tipos de regiões geométricas, tais como aquelas formadas por conjuntos disjuntos (desconexos).



- os neurônios de saída das redes PMC, quando aplicadas em problemas de Classificação de Padrões, realizam combinações lógicas, tais como AND e OR, das regiões que foram definidas pelos neurônios das camadas anteriores.

Perceptron Multi-Camadas : Classificação de Padrões



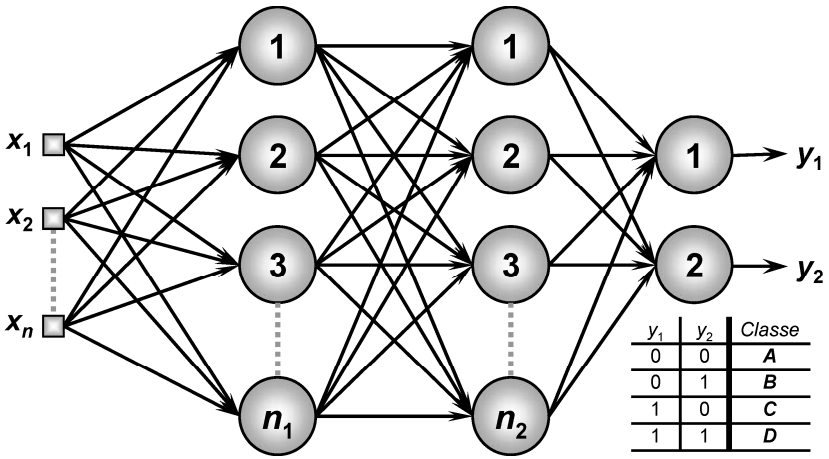
- Quantas camadas escondidas (no mínimo) terá o PMC?
- ii) Quantos neurônios (no mínimo) terá a primeira camada escondida?
- iii) Considerando-se as respostas dos neurônios da última camada escondida, qual função booleana que o neurônio de saída estaria então implementando?
- Por convenção, assume-se que as respostas dos neurônios da última camada escondida produzirão valor 1 para as suas sub-regiões internas (setinhas).

Perceptron Multi-Camadas : Classificação de Padrões



- Para problemas de classificação de padrões com mais de duas classes, há então a necessidade de se inserir mais neurônios na camada de saída da rede.
- Um PMC com apenas **Um neurônio** em sua camada de saída é capaz de distinguir somente **duas classes**.
- PMC composto de **Dois neurônios** em sua camada de saída poderia representar, no máximo, **quatro classes** possíveis.
- Generalizando, PMC com **m neurônios** em sua camada de saída seria capaz de classificar, teoricamente, até $\mathbf{2^m}$ **classes** possíveis

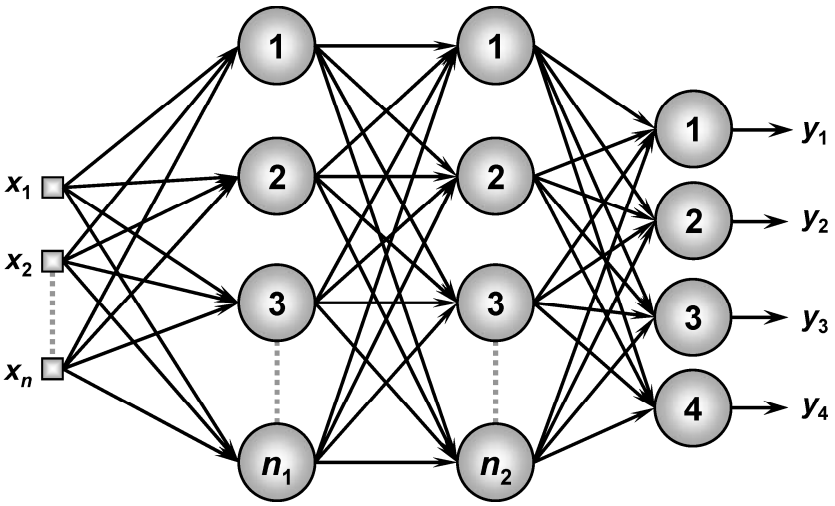
Perceptron Multi-Camadas : Classificação de Padrões



y_1	y_2	Classe
0	0	*A*
0	1	*B*
1	0	*C*
1	1	*D*

- **Codificação Multiclasses**: **<u>Sequencial</u>**
  - Em termos práticos, devido à eventual complexidade do problema, a adoção desta codificação pode tornar o treinamento bem mais difícil.
  - Neste caso, as classes estariam sendo representadas por pontos que estão espacialmente bem próximos entre si.
  - Tal situação poderia então demandar um incremento substancial no número de neurônios de suas camadas intermediárias

Perceptron Multi-Camadas : Classificação de Padrões



- **Codificação Multiclasses**: **One of C-Class**
  - É um dos métodos mais utilizados para codificação multiclasses.
  - Consiste de associar a saída de cada neurônio diretamente à classe correspondente.
  - Neste caso, a quantidade de neurônios na camada de saída do PMC será igual ao número de classes do problema.
  - Como exemplo, se houver 4 classes possíveis, as saídas do PMC seria também composta de 4 neurônios.

Perceptron Multi-Camadas : Classificação de Padrões



- **Pós-processamento**: Os valores produzidos pelos neurônios da camada de saída da rede PMC são números reais.
- Considerando os problemas de classificação de padrões, as respostas devem ser então pós-processadas, pois o que importa são aqueles valores discretos usados p/ rotular as classes.
- Para o caso da função logística, estes valores podem estar próximos de 1 ou próximos de 0 e desta forma, pode ser usada uma lógica adicional de processamento:

$$Y_i^{pos} \begin{cases} 1, se\ Y_i^{saida} \geq l^{sup} \\ 0, se\ Y_i^{saida} \leq l^{inf} \end{cases}$$

Referências



- RUSSELL, STUART J.. Inteligência Artificial. 2ª ed. São Paulo: Campus - Elsevier, 2004.
- FERNANDES, Anita M. R.. Inteligência Artificial: noções gerais. 1ª ed. Florianópolis: Visual Books, 2008. Capítulo 1.
- SILVA, I.N. – Sistemas Inteligentes – Notas de Aula da disciplina de Redes Neurais Artificiais – USP São Carlos, 2012.
- SILVA, I.N., SPATTI, D.H., FLAUZINO, R.A., Redes Neurais Artificiais para Engenharia e Ciências Aplicadas. 1ª ed. São Paulo: Editora Artiliber, 2010.
- NASCIMENTO JUNIOR, Cairo L., YONEYAMA, T. Inteligência Artificial em Controle e Automação. 1ª ed. São Paulo: Edgard Blücher, 2000.


Facens
AQUI TEM ENGENHARIA